



AGILE METHODOLOGIES AND SCRUM

R 251003

Contributors (in alphabetical order)

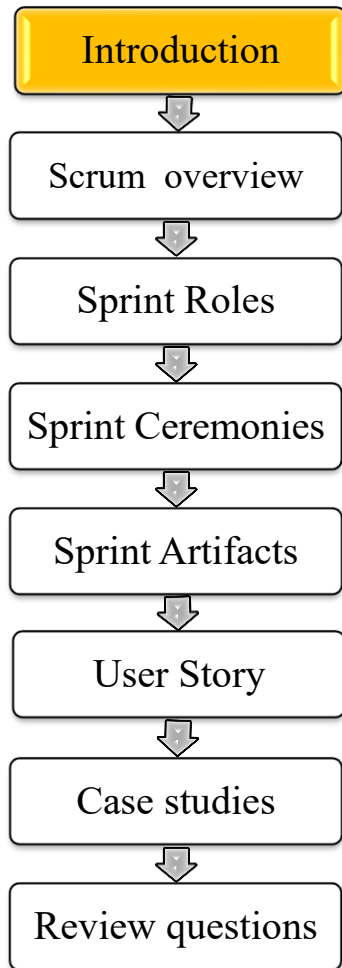
Alessandro Ceseno, Gianmario Motta, Nicola Sfondrini

Editor

Gianmario Motta, motta05@unipv.it



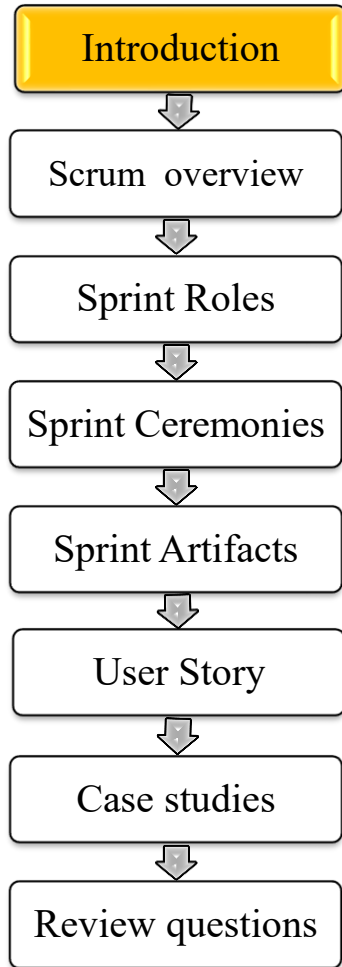
The Agile Project approach



- “Project approach” is the way of running the project life cycle
- A Project Approach can be
 - Incremental
 - Predictive
 - Hybrid
- Agile is an Incremental Project Approach



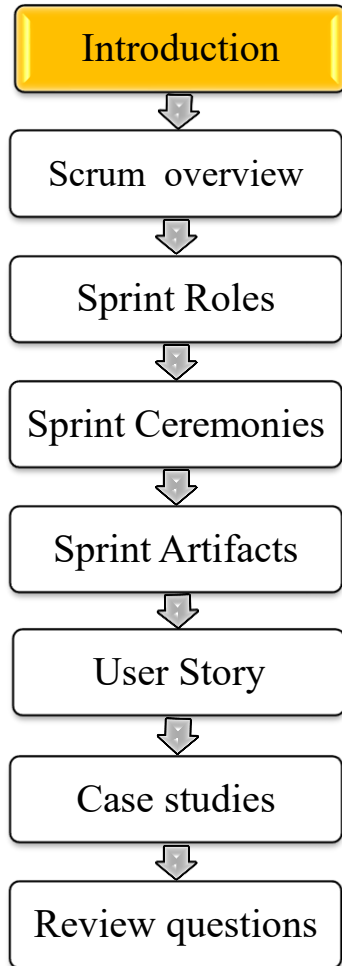
Why Agile?



- For many decades, software projects have been relying on a rigid top-down approach where:
 - The software product is realized top-down through a series of phases
 - The analysis phase relies on abstract modeling notations, such as UML, BPMN, and alike
 - To preserve the product integrity, no substantial change is allowed during the project
 - A large amount of documentation is produced for both products (e.g., Class diagrams, Use-Case diagrams, etc.) and management (e.g., plans, reports)
 - The whole project may last many months (even years) so that the released product may result obsolete



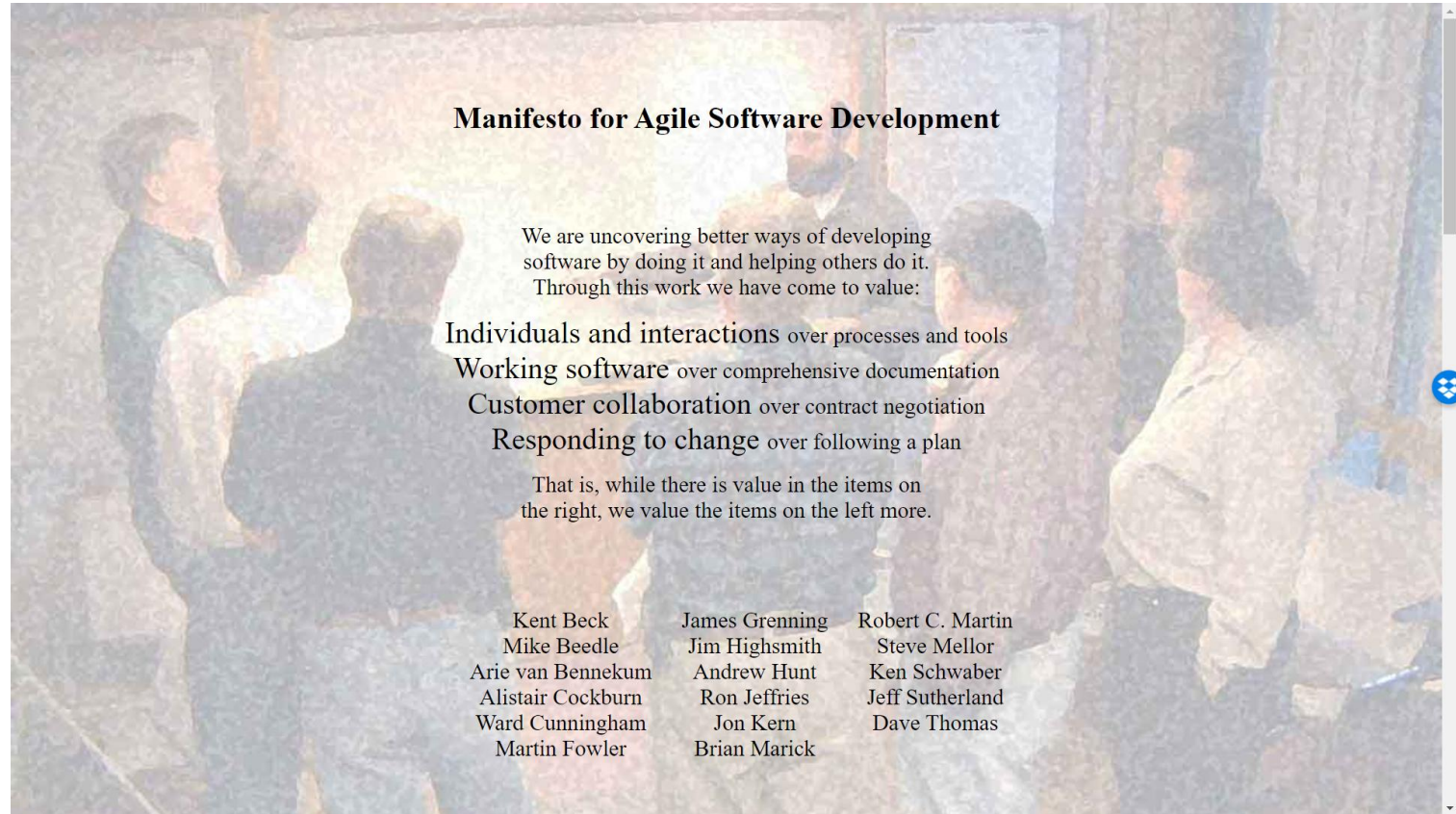
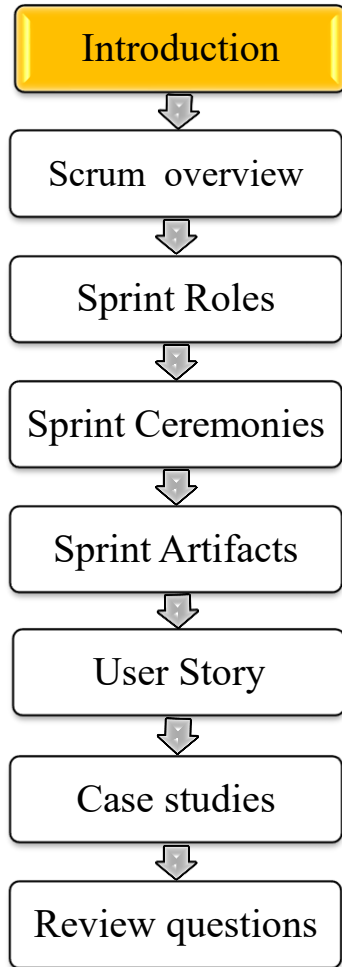
Why Agile?



- The Agile approach is a reaction to the bare top-down approach and is intended to simplify projects by:
 - Short delivery: weeks instead of months or years
 - Face-to-face interaction instead of paper reports
 - Easy changes of features even during the project instead of frozen changes
 - Visual project progress: Kanban instead of PERT or Gantt
 - Team based on a peer collaboration instead of hierarchy
 - Easy user requirements (User Stories) instead of modeling languages such as UML and /or Object-oriented modeling and/or BPMN
- Not surprisingly, Agile is the most popular approach for software application projects

The Agile Manifesto

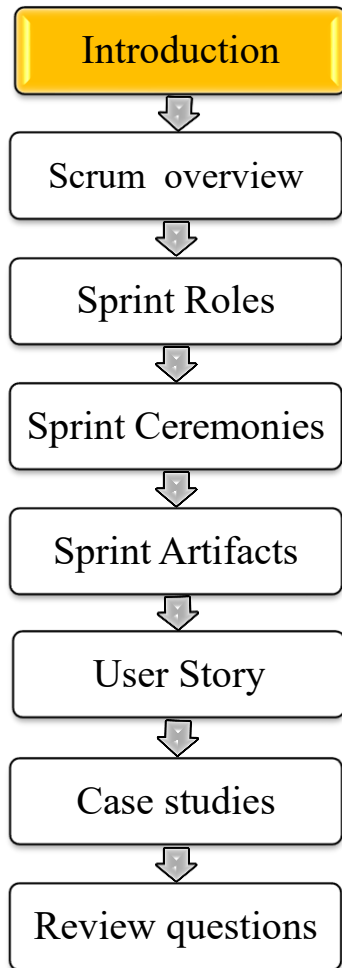
<https://agilemanifesto.org>



Agile methodologies are based on the Agile Manifesto issued in February 2001 at Snowbird, Utah, USA, by independent experts



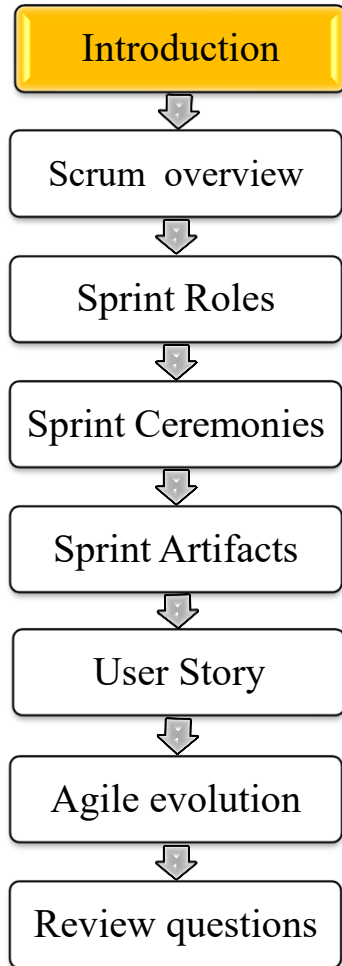
Excerpts from the Agile Manifesto



- *We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*
 - *Individuals and interactions over processes and tools*
 - *Working software over comprehensive documentation*
 - *Customer collaboration over contract negotiation*
 - *Responding to change over following a plan*
- *Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.*
 - *Welcome changing requirements, even late in development. Agile processes harness (= exploit) change for the customer's competitive advantage.*
 - *Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.*
- *Working software is the primary measure of progress.*

The Agile Manifesto

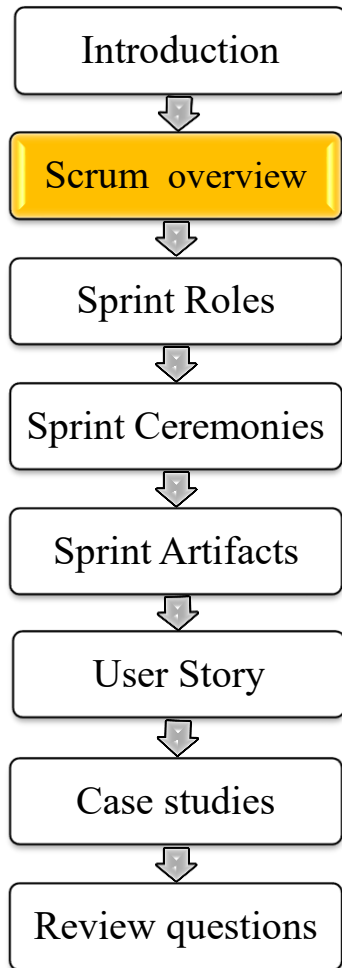
<https://agilemanifesto.org>



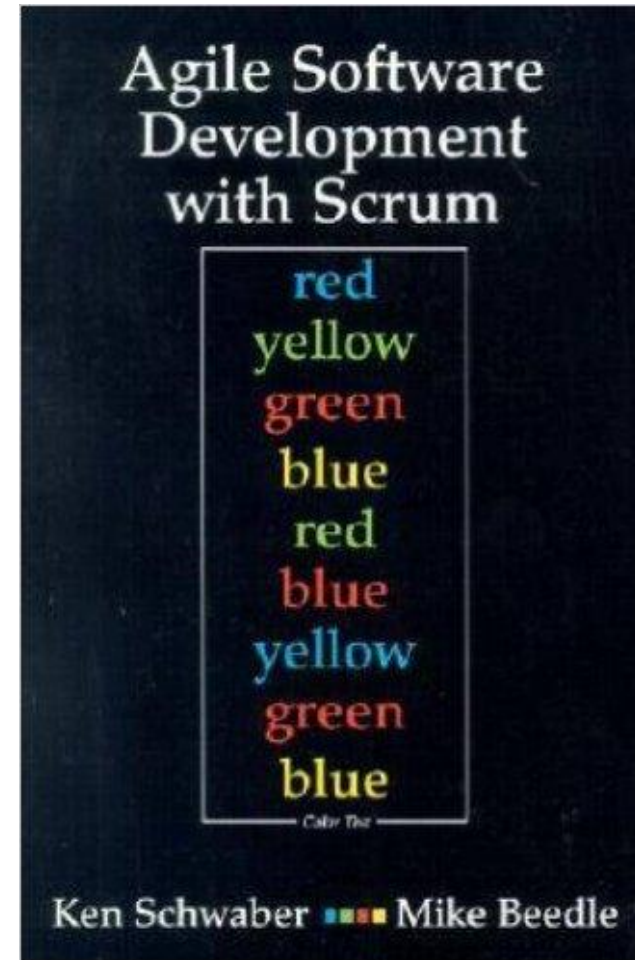
1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Businesspeople and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity--the art of maximizing the amount of work not done--is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly,



SCRUM

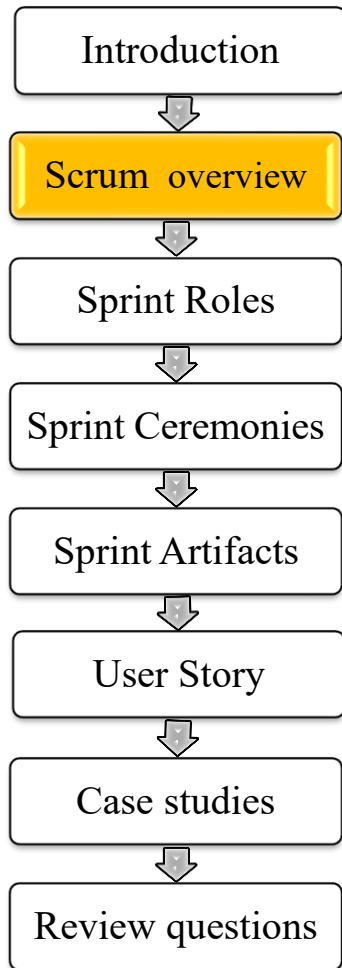


- SCRUM (= a clash in a rugby match) is an Agile software development framework, based on small teams working in an intensive manner, as opposed to a sequential phased approach (“waterfall”).
- SCRUM is by far the most popular Agile methodology

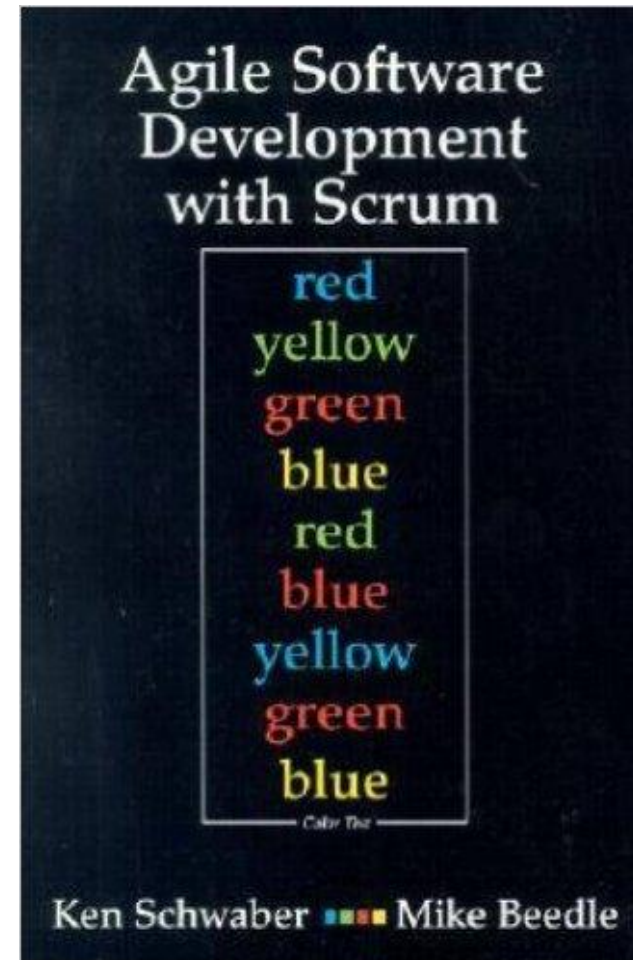




SCRUM story (from Wiki)



- 1986: Hirotaka Takeuchi and Ikujiro Nonaka introduce the term “Scrum” in an article on the New Product Development Game.
- 1995: Sutherland and Schwaber present a paper on the Scrum methodology at OOPSLA '95 conference in Austin, Texas.
- 2001: Schwaber with Beedle describes Scrum in the book “Agile Software Development with Scrum”.
- 2002: Schwaber with others founds the Scrum Alliance and sets up the Certified Scrum Master program and its derivatives.
- 2009: Schwaber leaves the Scrum Alliance and founds Scrum.org.



SCRUM: the original diagram

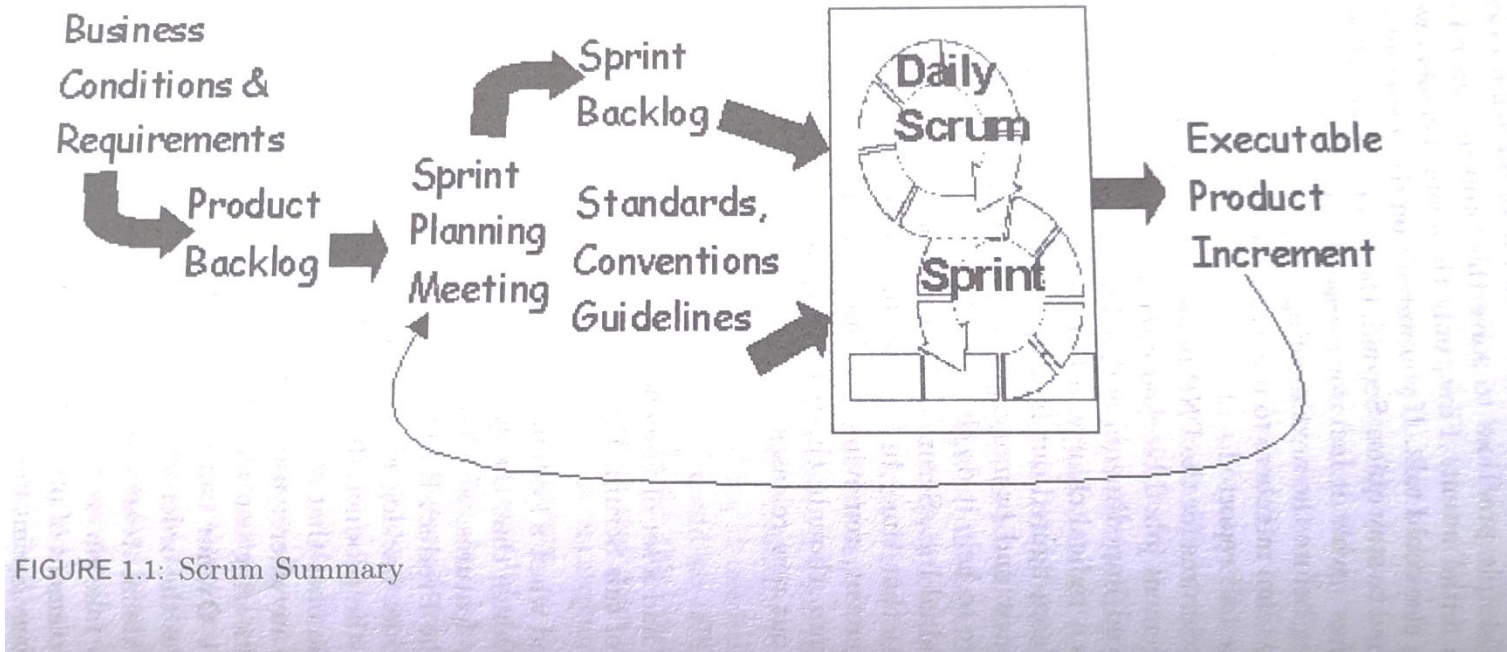
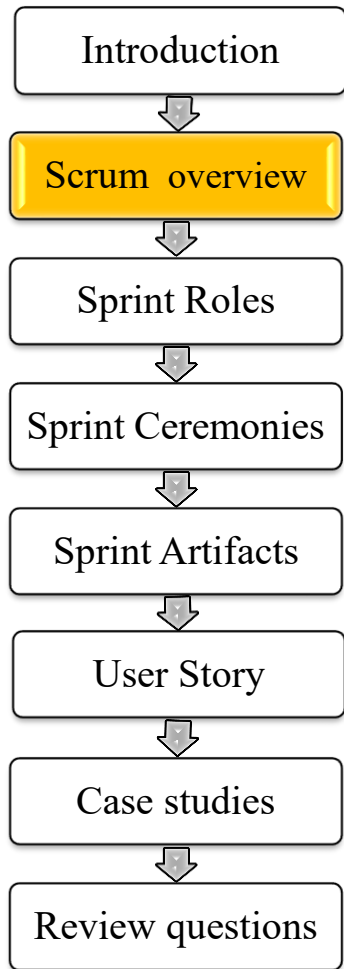
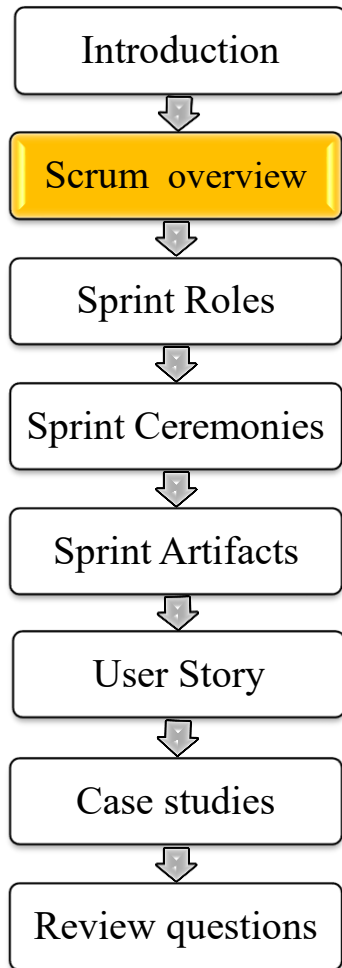


FIGURE 1.1: Scrum Summary

Schwaber K., Beedle M., Agile Software development with Scrum (Prentice Hall, 2011) p.8, fig. 1.1

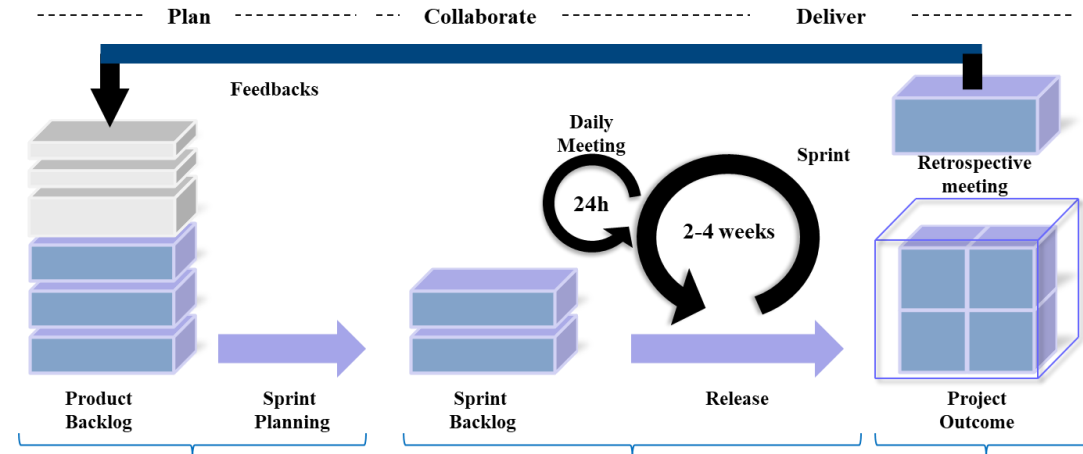
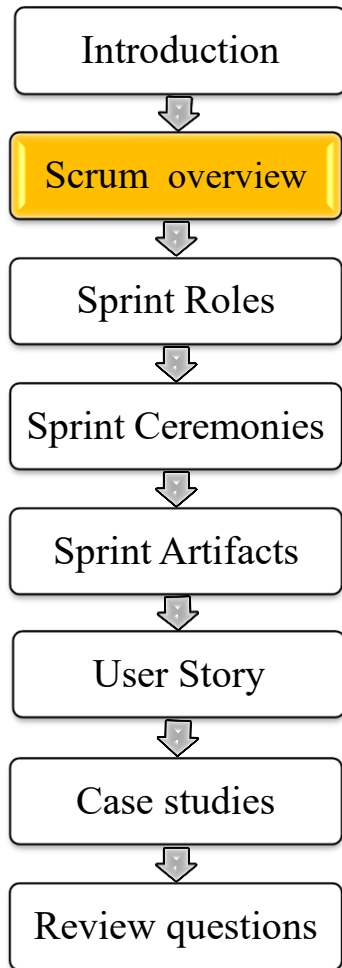
SCRUM



- In the following pages we illustrate the key concepts of Scrum and Agile:
 - Scrum’s Sprint framework
 - Roles
 - Ceremonies
 - Artifacts
 - User Stories, a requirement statement that perfectly fits the Sprint framework



The Sprint framework: overview



The Sprint cycle is the core of SCRUM

A Sprint lasts 15-30 days

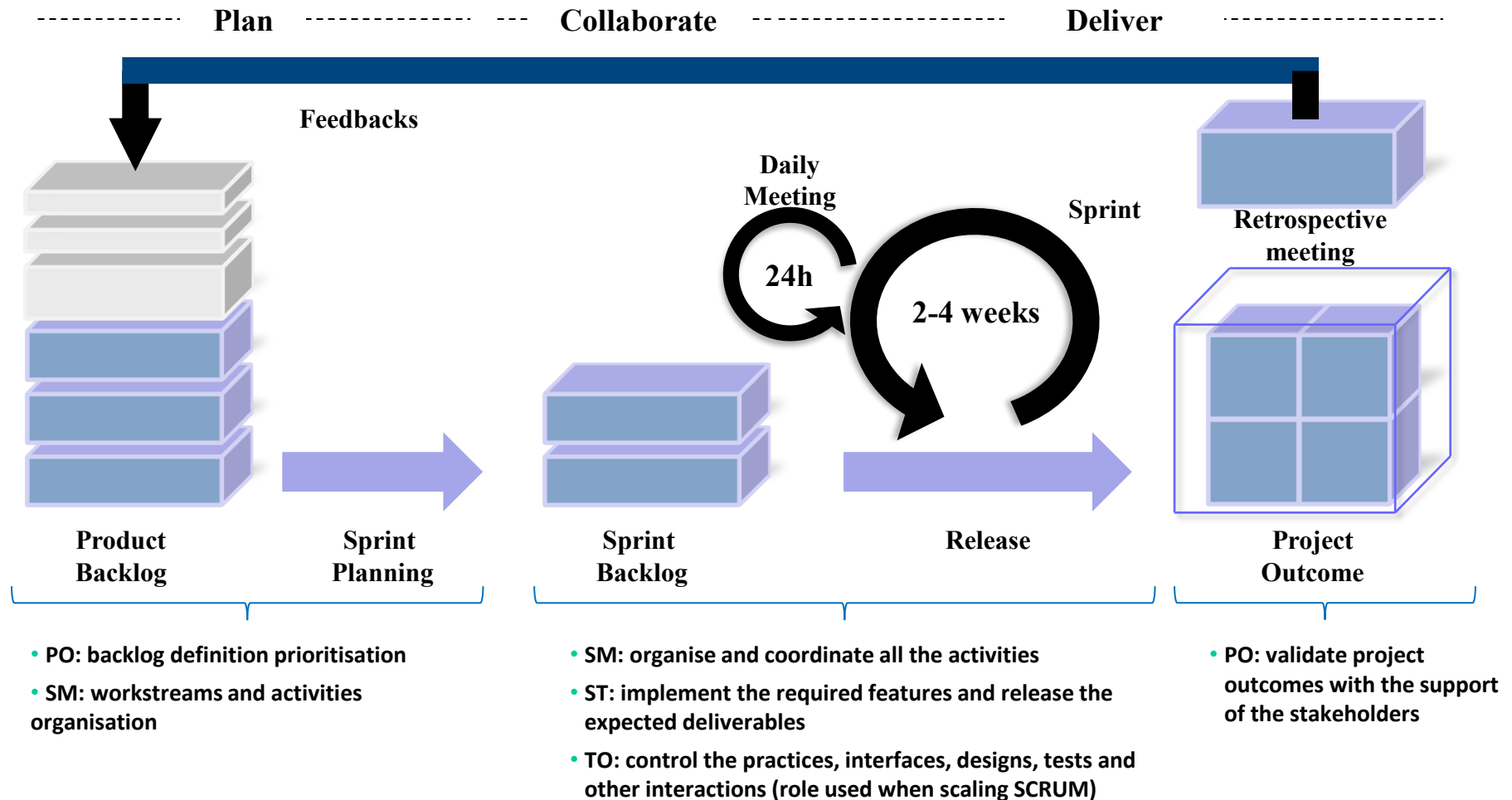
Each Sprint delivers a usable software product that is developed from a backlog

A Sprint team is made of 3-6 software development EXPERTS

Since a typical Sprint is performed by a team of 5 people and lasts 15-30 days, a project with 5 Sprints will require 5×5 month-person = 25 month-person = 2,5 FTE (Full Time Equivalent)



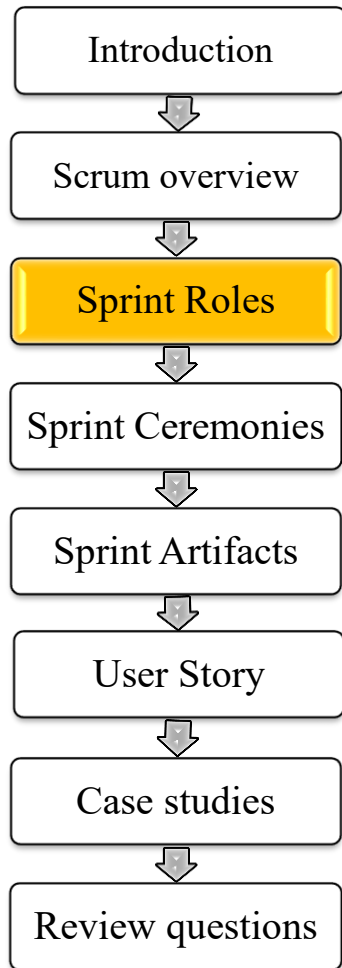
The Sprint framework: overview



PO = Product Owner | SM = SCRUM Master | ST = SCRUM Team | TO = Technical Owner



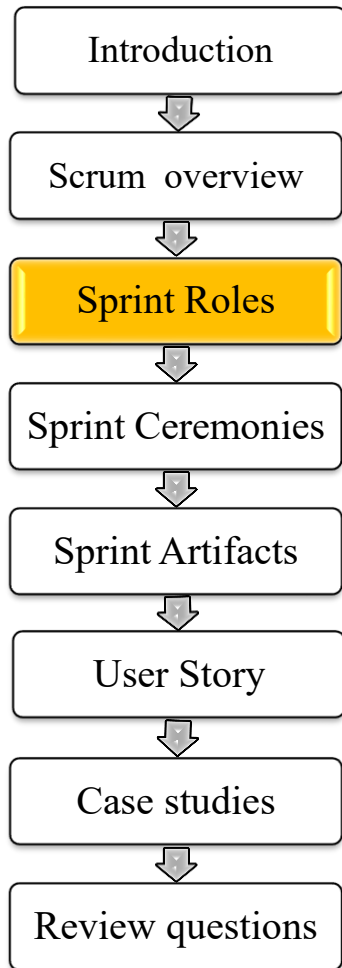
Sprint roles



- **Product Owner (PO)**
 - Responsible for the ROI of the product development effort
 - Defines programme scope, goals and revises release plan
 - Constantly re-prioritises the activities and defines the product backlog
 - Approves the outcomes of each sprint
- **Scrum Master (SM)**
 - Facilitates the Scrum process & collaboration among the different stakeholders
 - Keeps the Scrum artifacts visible and facilitate Team self-organization
 - Helps to resolve impediments
 - Defines mitigation actions to fix possible issues and risks
 - Shields the team from interference and pushes improved engineering practices
- **Scrum Team (ST)**
 - The team is self-organized, and it is cross-functional
 - Develops the features required
 - Tests and deploys each single brick in production environment
 - Incident monitoring and performance evaluation
- **Technical Owner (TO)**
 - Parallel to PO, controls practices, interfaces, designs, tests,
 - Interaction issues (e.g., consistency of estimations, to “synchronize” integrations)
 - Responsible for integration issues among Scrum teams (e.g., coding standards)



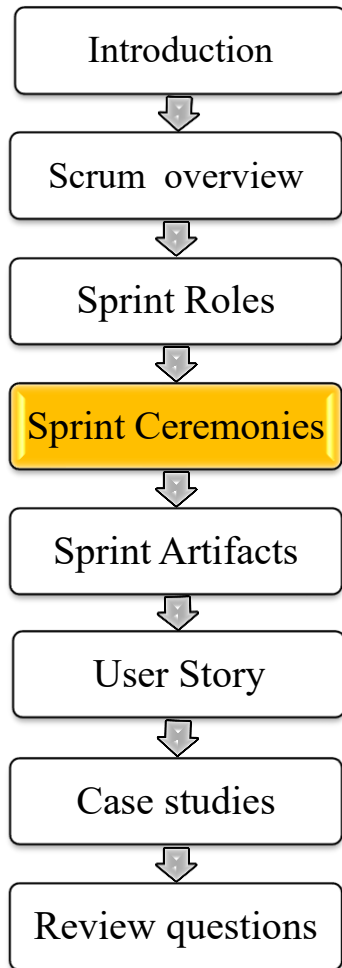
Sprint roles: Stakeholders



- Stakeholders are actors who are involved/affected by the project but are not working in the project
- Stakeholders can be
 - Internal: Employees, Managers
 - External: Customers, Suppliers, Government
- The PO shall manage the relations with stakeholders, to make sure that the project is accepted
- The impact of the project on stakeholders may be high or low
- Accordingly, the PO shall select the appropriate relational approach – formal/informal committee, demo, formal/informal meetings

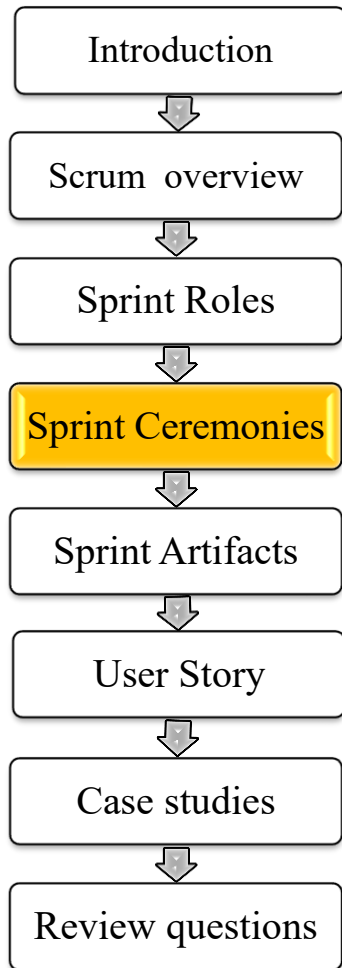


Sprint Ceremonies: Sprint Planning

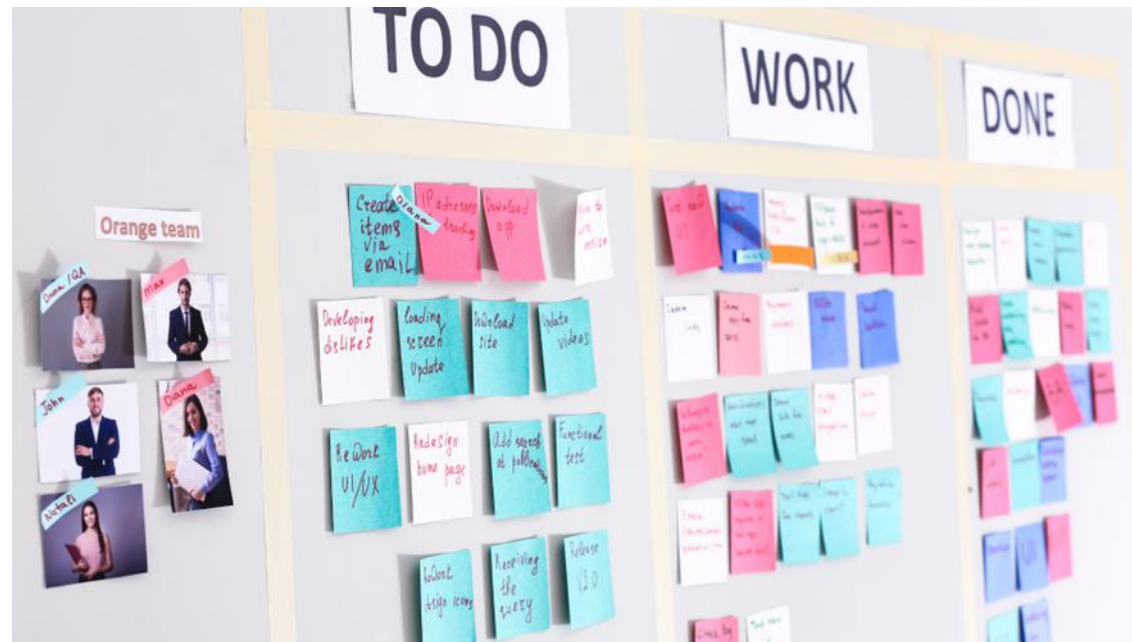


- Sprint Planning lasts one day
- The Product Owner and the Scrum Team meet and define the Sprint Backlog by selecting the User Story to be processed
- The Scrum team identifies the Tasks of each User Story (a User Story includes multiple tasks)
- The Sprint Planning works out a Sprint Board that lists the User Stories to be processed and the related tasks.
- The Sprint Board is a grid with 4 columns, namely
 - TODO
 - IN PROGRESS
 - BLOCKED
 - DONE
- Sprint progress is shown by simply posting the task card from one column to another (e.g., using post-its)

Sprint Ceremonies: Sprint Planning by Sprint Board



Example of board with photos and names of team members



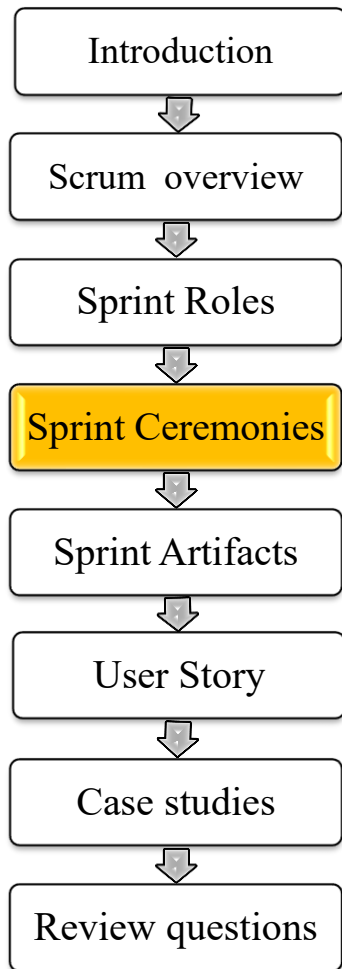
SOURCE: created with <https://www.canva.com/>



Sprint Ceremonies:

Sprint Planning by Sprint Board

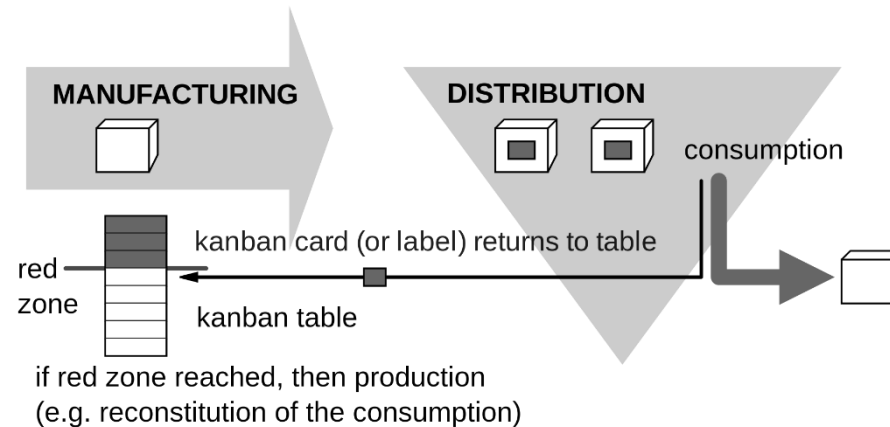
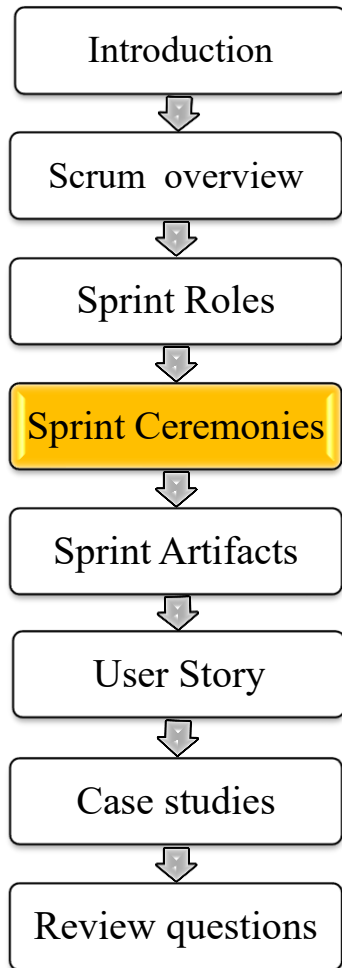
Example of board with post-it



SOURCE: created with <https://www.canva.com/>

Sprint Ceremonies:

Sprint Planning: the Kanban concept

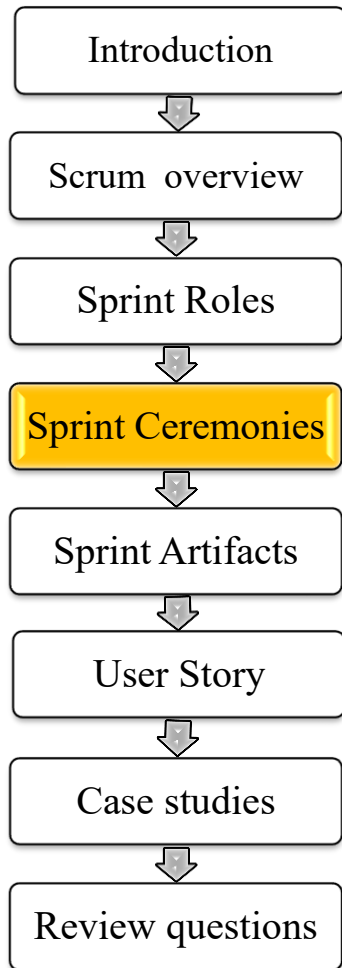


- The Sprint board stems from Kanban
- Kanban (= signboard or billboard) is a scheduling system for lean manufacturing that takes its name from the cards that track production within a factory (also called just-in-time manufacturing, JIT)

Credit: <https://en.wikipedia.org/wiki/Kanban> as of May 23^o, 2022



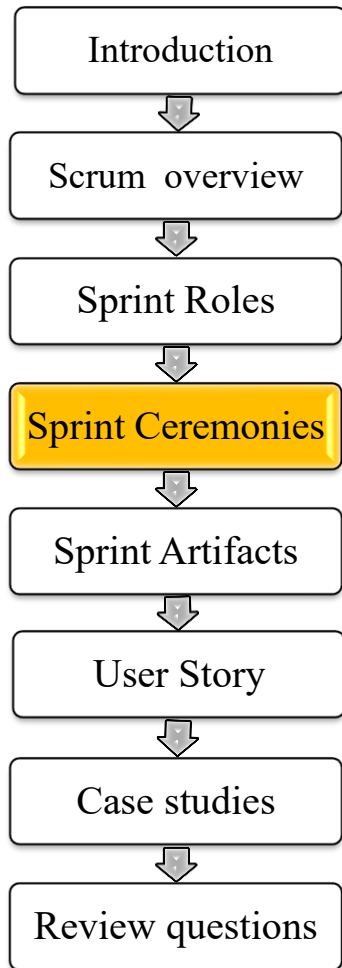
Sprint Ceremonies: Stand-up meeting



- Every day, during the execution of the Sprint, the Scrum Master facilitates the Daily Stand-up meeting with the Team.
- During the Stand-up meeting, each team member briefly answers these questions:
 - What did I do yesterday?
 - What (if anything) is blocking my progress?
 - What will I do today?
- The meeting is done every morning and is usually timeboxed in 15 minutes; it runs in the same place (physical or / and virtual) and at the same time.
- *The Stand-up meeting is for the Team. The presence of the Product Owner is optional. The Scrum Master may attend in case of problems.*



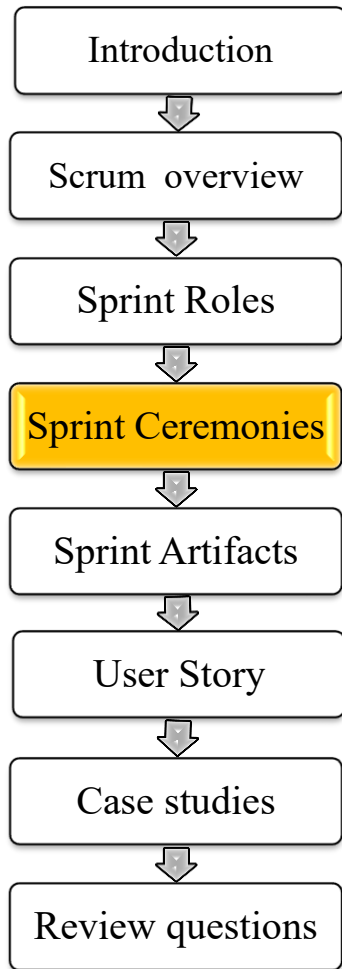
Sprint Ceremonies: Stand-up meeting



Source: Klean Denmark - Daily Sprint Meeting, CC BY-SA 2.0,
<https://commons.wikimedia.org/w/index.php?curid=19838566>



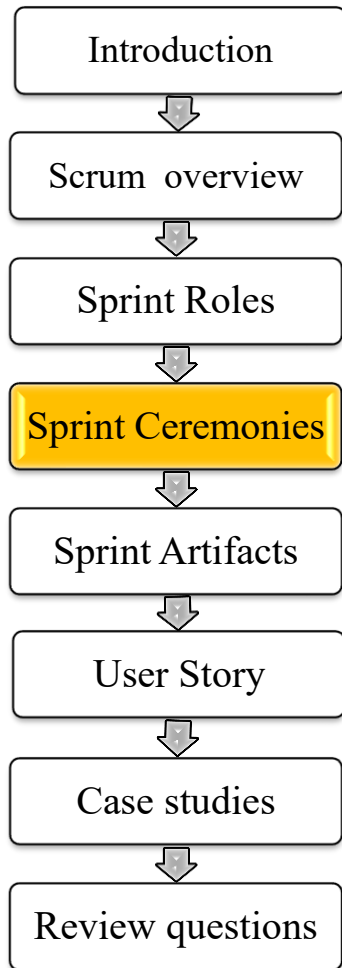
Sprint Ceremonies: Sprint Review



- Sprint Review is a meeting held at the end of each Sprint and typically lasts one hour or so; it is attended by the Product Owner, Scrum Master, Scrum Team, Stakeholders.
- In the Sprint Review:
 - The work done is demonstrated and a feedback is received from the Stakeholders.
 - The Product Owner validates or invalidates the User Stories that have been worked in the Sprint.
- The most frequently questions in the Sprint Review concern the correct implementation of the product, the progress of the product, the product compliance with user's expectations



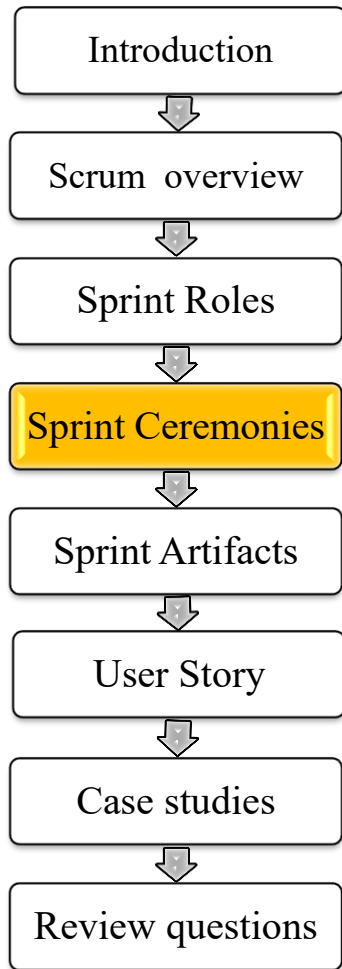
Sprint Ceremonies: Sprint Review



Source: <https://concisesoftware.com/blog/agile-essential-scrum-meetings/>



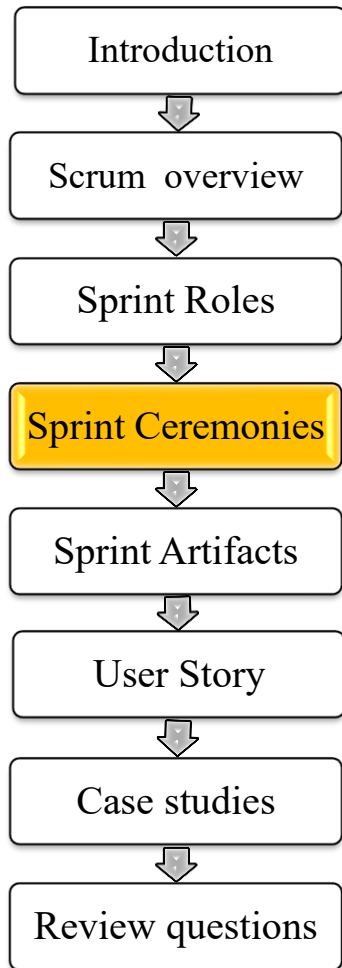
Sprint Ceremonies: Retrospective Meeting



- At the end of the Sprint, the Scrum Master facilitates the Retrospective, where:
 - the team reflects on the lessons learned and decides on the actions (typically 1 to 3) to be taken in the next Sprint.
 - a team member takes the responsibility of these actions
- Popular retrospective frameworks are:
 - Glad, Mad, Sad Retrospective
 - Starfish Retrospective.
 - Textual Retrospective (see figure)

Sprint Ceremonies: Retrospective Meeting

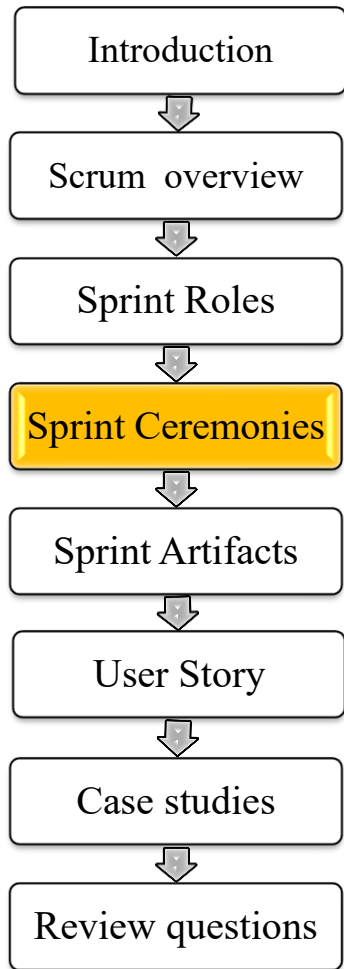
Starfish Retrospective



created with <https://www.canva.com/>

Sprint Ceremonies: Retrospective Meeting

Glad Sad Mad Retrospective



Retrospective Glad Sad Mad



What Made You
Glad?



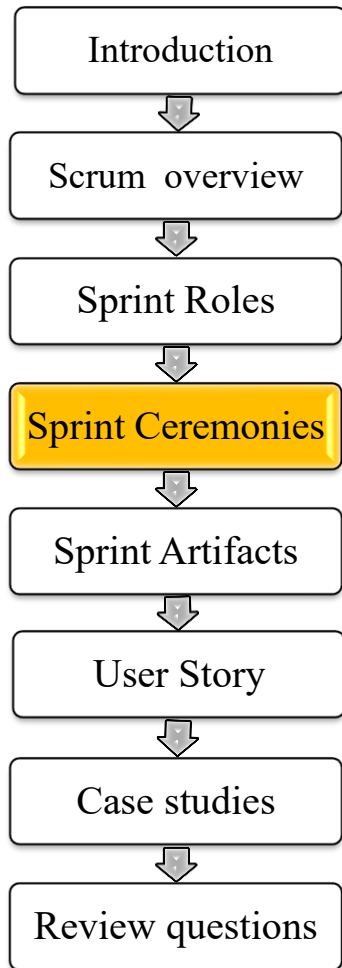
What Made You
Sad?



What Made You
Mad?

Sprint Ceremonies : Retrospective Meeting

Textual Retrospective



Retrospective

Date:
Project Milestone:

- 1 Identify the project milestone to review, and recap the highlights briefly.
- 2 Look back on what you did well as a team. Copy and paste the sticky notes to add individual ideas under "What was good?"
- 3 Group similar ideas together and discuss. Repeat steps 1 & 2 for "What was bad?"
- 4 Brainstorm ideas to improve problem areas. Translate them into action items.

What was good?

10 MINUTES

Our team did really well with...

What was bad?

10 MINUTES

We can improve on...

Ideas

10 MINUTES

We can try...

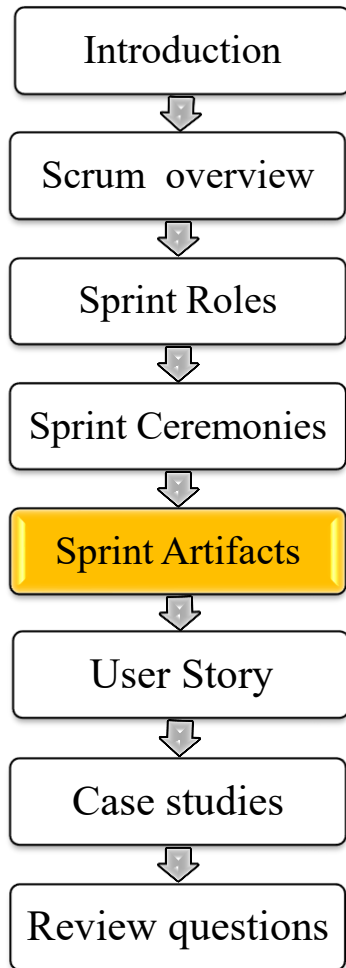
Actions

10 MINUTES

Action item:
Owner:
Due date:



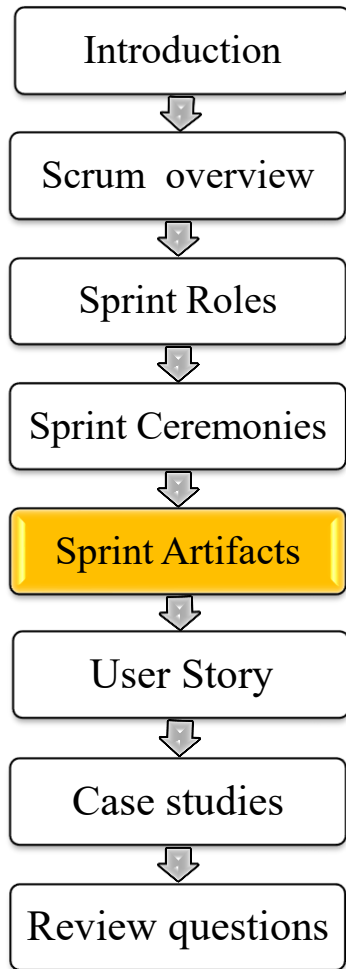
Sprint Artifacts



- Various artifacts are used in Scrum Sprints
 - Product Backlog
 - Sprint Backlog
 - DOR Definition Of Ready
 - DOD Definition Of Done
 - Product Increment
 - Impediment List
 - Burndown Chart
- These artifacts reflect a visual model, where information is displayed on boards that everybody can see and, also, are manually created
- In our view, consistently with Scrum philosophy, artifacts should be the fewest, and, ideally, should include only:
 - Product Backlog
 - Sprint Backlog
 - Burndown chart (optional)



Sprint Artifacts templates (example)



Product Backlog Items



Sprint Check List



MoMs



Document

Sprint Task List



Burn Up/Down Charts



Impediments List



Timelines



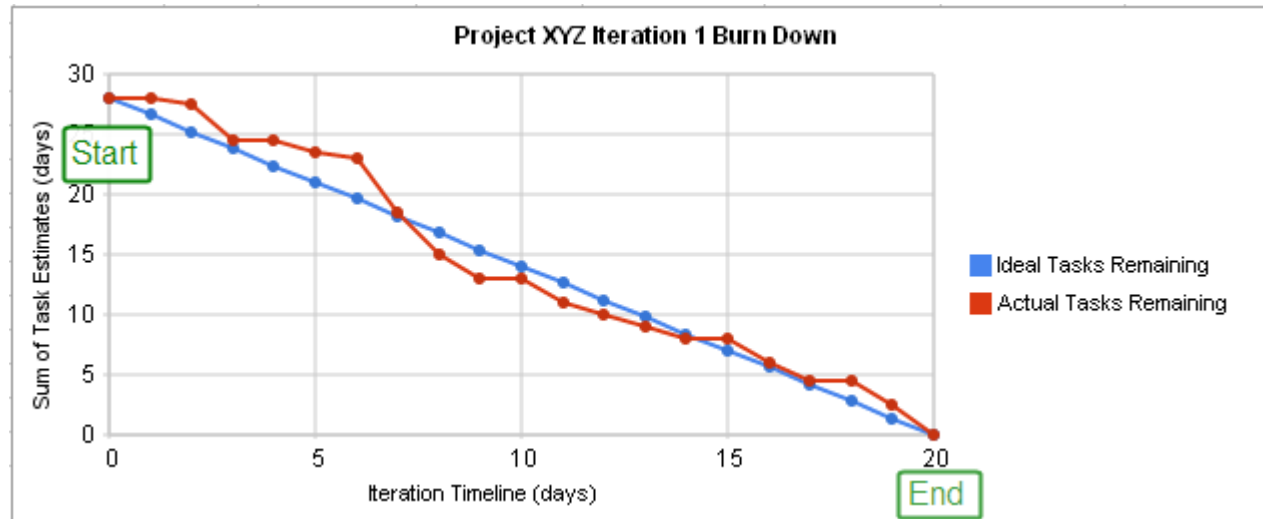
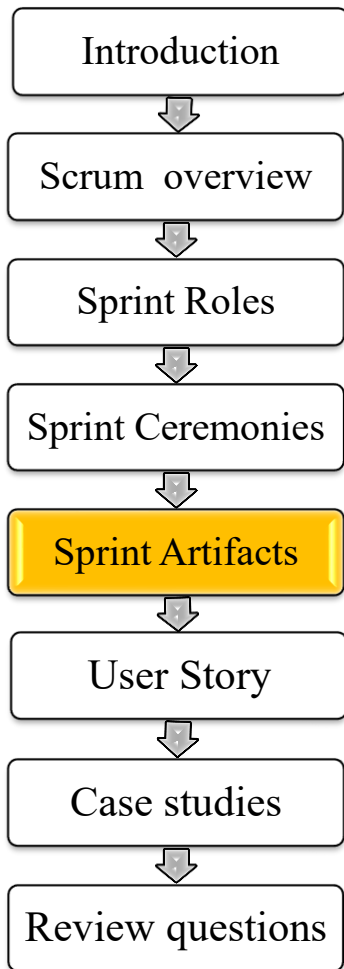
Presentation

Satisfaction Histograms





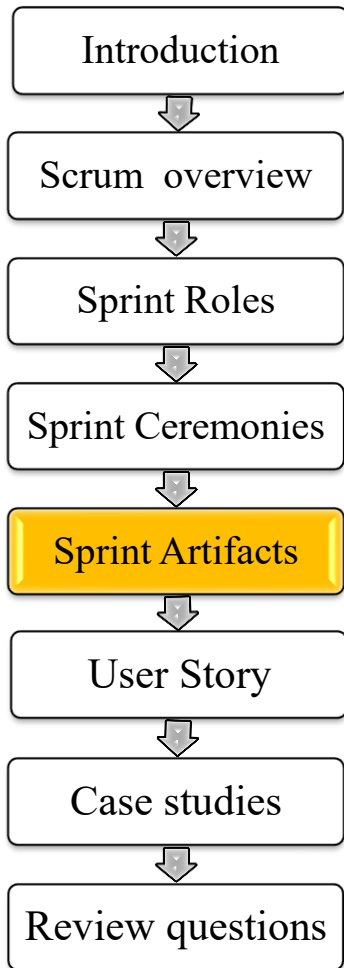
Sprint Artifacts: Burndown Chart



- Burndown Chart is a way to display the progress; updated daily, it displays the total work remaining within a Sprint
- It may be used as a report, inviting intervention by management outside the Team, thus becoming an impediment to self-organization.

Sprint Artifacts:

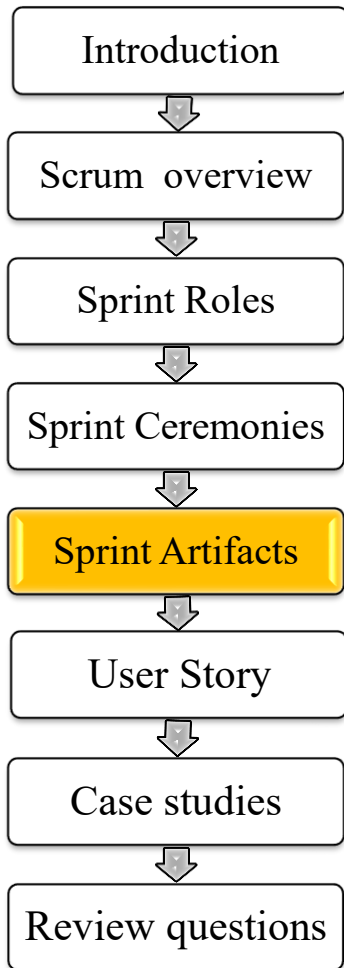
Product Backlog and Sprint Backlog



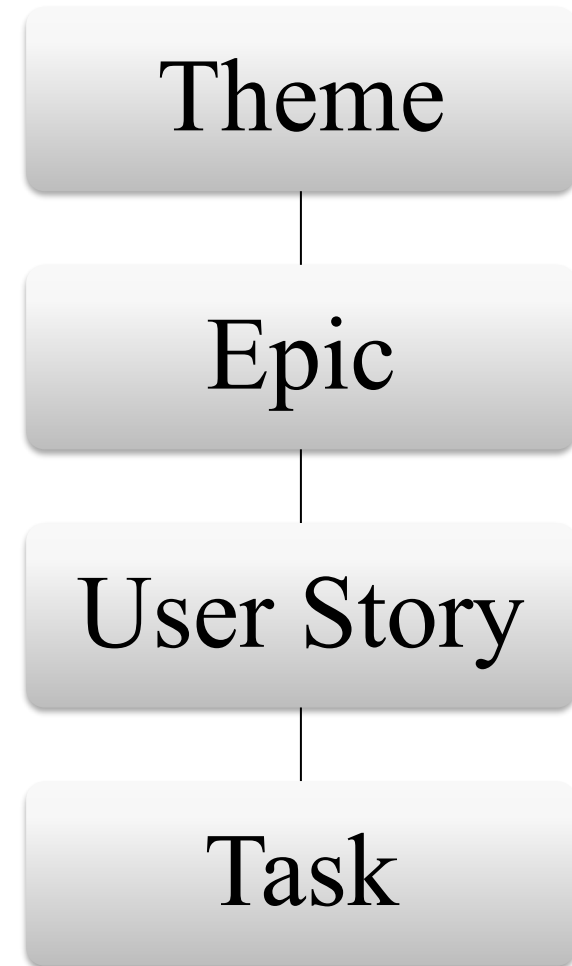
- The Product Backlog is the set of features that a project shall realize
- The Product Backlog is built in a series of steps at Project Level (Product Backlog) and at Sprint Level (Sprint Backlog)
- At Project Level:
 - At Project start, the Product Owner identifies a set of desired features, called "Epics", i.e., macro functionalities, which can be grouped into "Themes".
 - The Product Owner, by sequencing Themes and Epics, defines a Roadmap, to which Sprints are associated.
- At Sprint Level:
 - When the first Sprint begins, the Product Owner decomposes the Epic into User Stories.
 - A set of User Stories are assigned to the first Sprint (Sprint Backlog)
 - At the end of each Sprint, the process is repeated until the Backlog is empty (or blocked)

Sprint Artifacts:

Product Backlog and Sprint Backlog

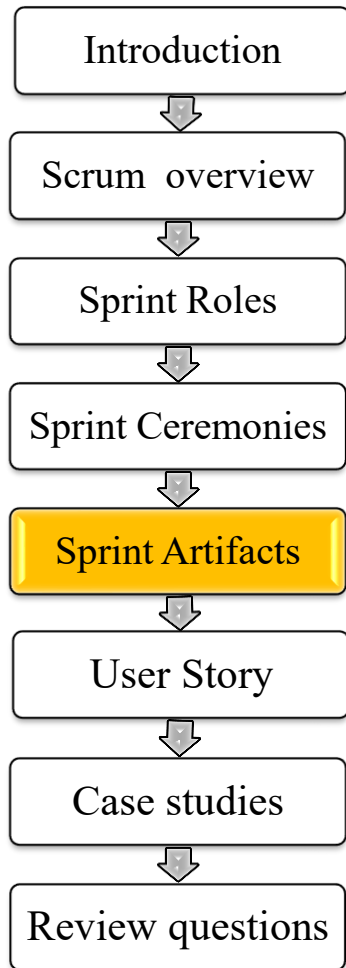


- Product and Sprint backlogs describe the product at different detail levels
 - Theme
 - Epic
 - User Story
 - Task
- We here illustrate their main characteristics

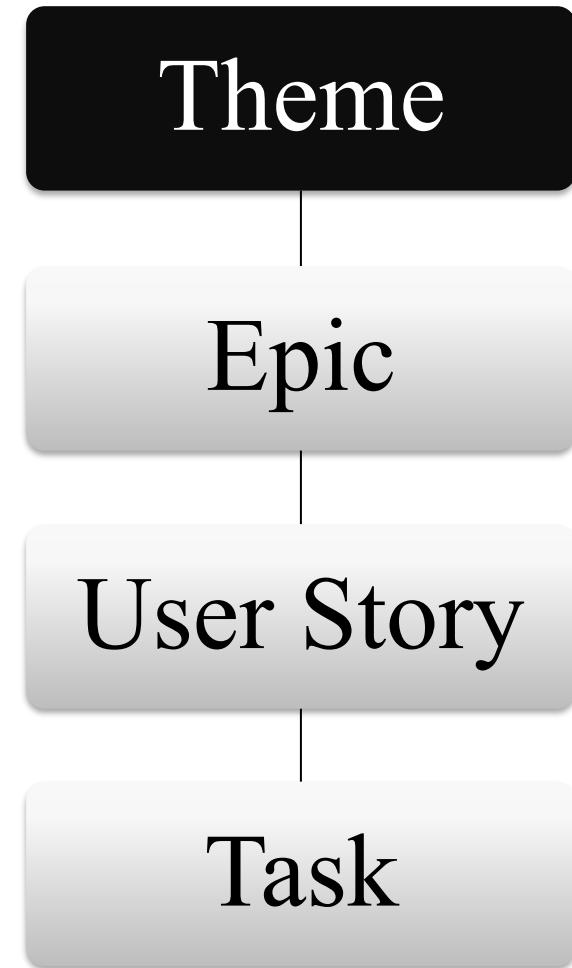


Sprint Artifacts:

Product Backlog and Sprint Backlog:

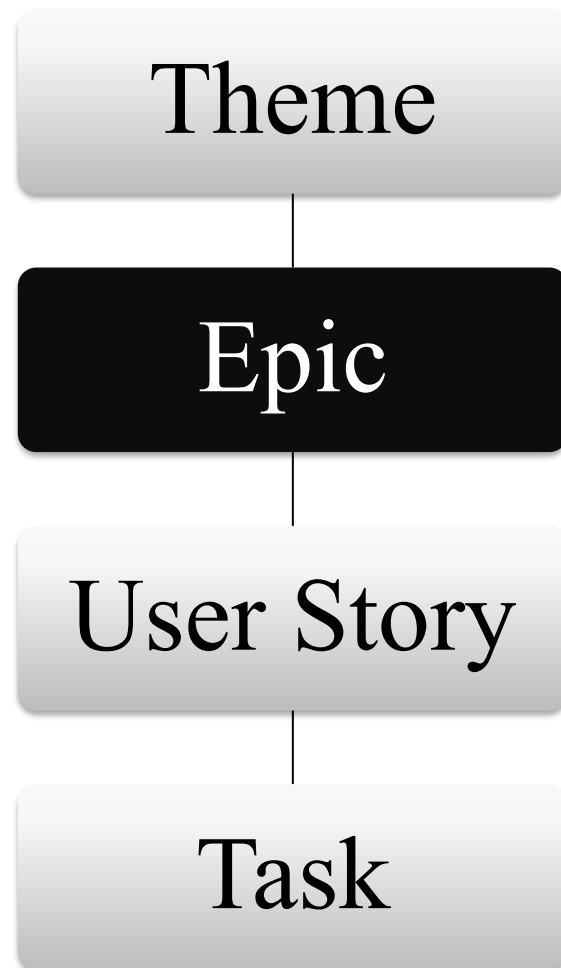
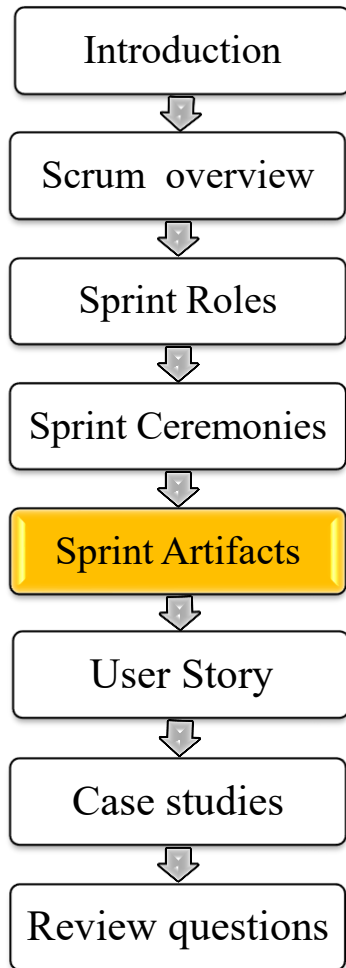


- Theme
 - It is the top level of backlog and describes:
 - tangible products, such as a trading application
 - abstract goals, such as performance tuning
 - A product owner breaks down a Theme into one or more Epics.



Sprint Artifacts:

Product Backlog and Sprint Backlog

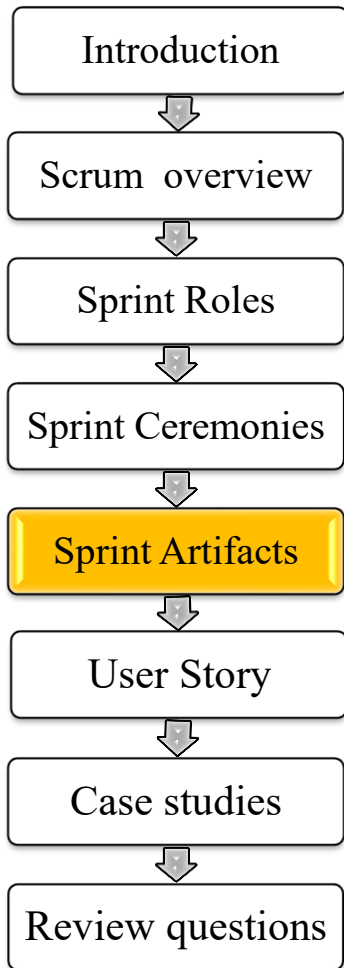


- Epic
 - It is a part of a Theme
 - It typically groups requirements not yet rationalized into stories.
 - An Epic can be also built bottom-up by grouping a set of related user stories

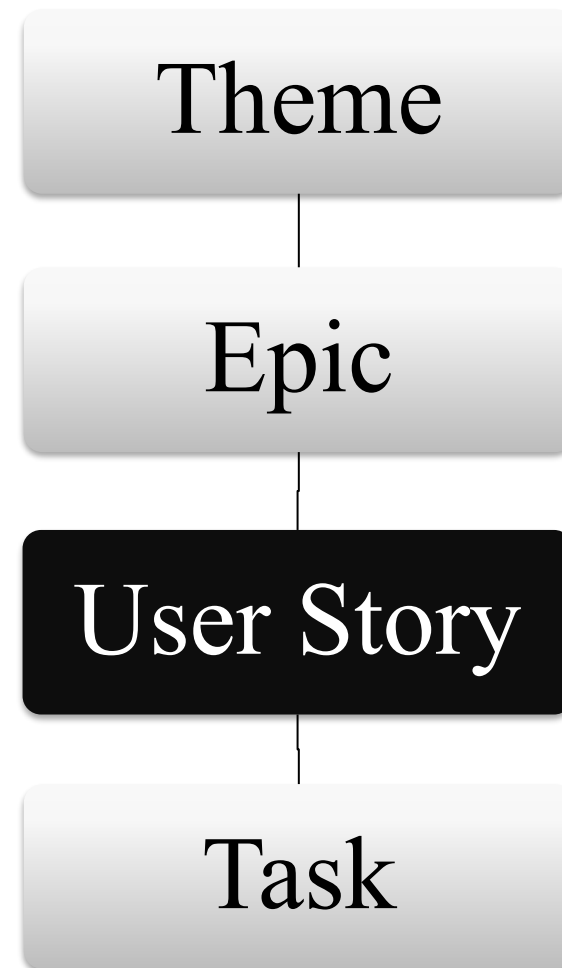


Sprint Artifacts:

Product Backlog and Sprint Backlog

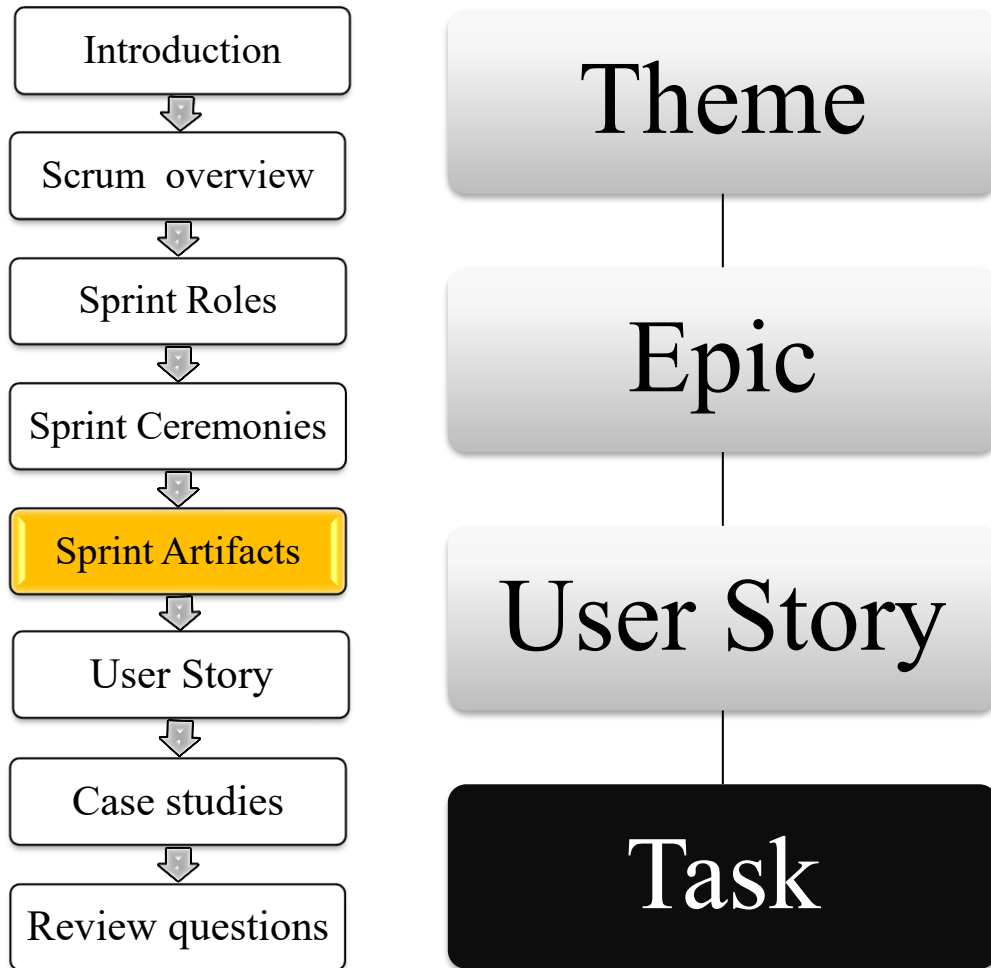


- A User Story is
 - a requirement written in business language according to a predefined structure
 - created by the Product owner
 - should be small enough to be completed in one Sprint
- The effort required to complete a User Story is estimated in Story Points by SCRUM team members, based on the individual experience and complexity of the tasks to accomplish: the more points the more effort.



Sprint Artifacts:

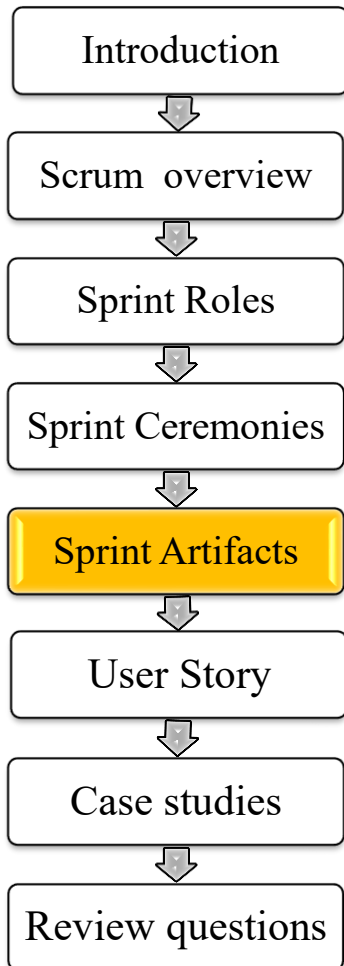
Product Backlog and Sprint Backlog



- Tasks:
 - are discreet pieces of work required to complete a Story:
 - typically require 4-12 hours
 - are chosen by team members
- Time-effort for each task is tracked & reported in the burn down chart
- If the planned hours for a task exceed an agreed period of time, such as eight hours, the task can be split into smaller tasks
- A story is complete when all tasks are complete



Sprint Artifacts: Sprint Backlog (example)



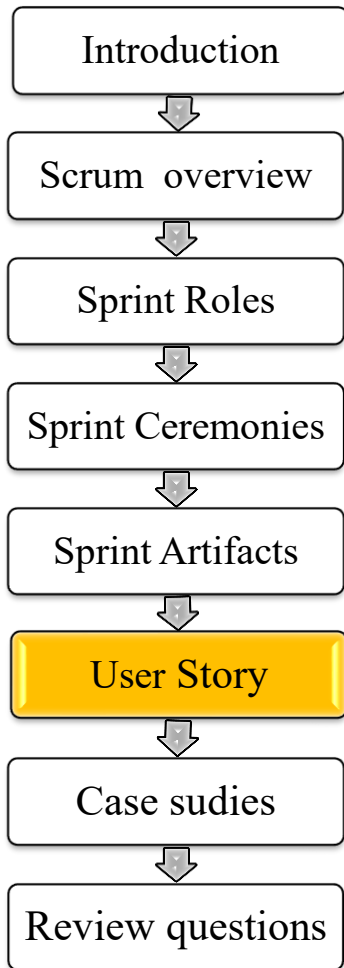
Story	Waiting		Done
	Waiting	Working 2 / 3	
+ add task	+ add task	+ add task	+ 20 archived tasks + add task
2 As a Marketing Data Analyst, I need to run the analytics reports the so that I can build the monthly media campaign plans.	0 AX PO 5687 - Office Supplies 1 GXO services -19 PO 5685 - Office Supplies 0 PO 5684 - Office Supplies 0 IT repair charge 834 0 Budget Req 56D -16 PO 56998 - IT order 45	786 Contracting Bill -1 Electric Bill	PO 98023 - productivity software AX MDM services DC PO 56991 - IT order 39 Reimbursement request 9385 JD Meank Bill 45 Reimbursement request 9384 Budgeting request 89000 PO 94022 - reporting software
-11 As an EBC Requester, I want to know what catalog of services TYTREE IT offers so that I can determine if an existing platform can support my proposed project.			
2 As an CF Requester, I want to have a checklist of outputs so that I can submit a complete CF request.			
1 As a user, I can backup my entire hard drive.			

The Sprint Backlog is made of User Stories split in Tasks

The Sprint Board displays tasks that are either to-do (waiting) or in progress (working) or done .



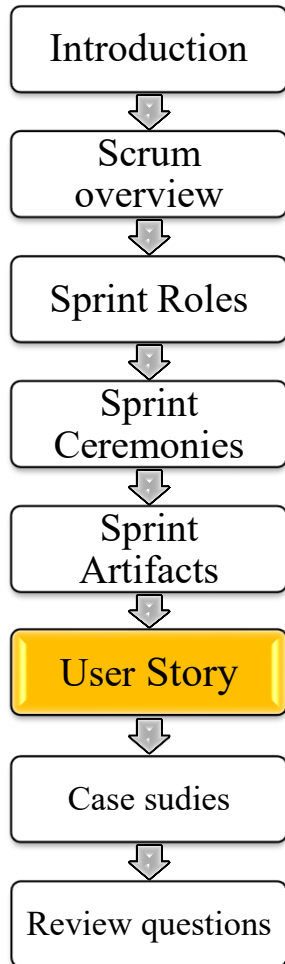
User Stories



- The principles of User Stories were generalized in 2004 by Mike Cohn in "User Stories Applied: For Agile Software Development"
- The User Story is a template where requirements are expressed by the statement
 - *As a (role)*
 - *I want (feature)*
 - *So that (benefit)*
- User Stories are written in business language.
- User Stories shall include **acceptance tests** which can be written in technical language.



User Stories Examples



- As a Marketing Data Analyst
- I need to run the analytics report
- So that I can build the monthly media campaign plans

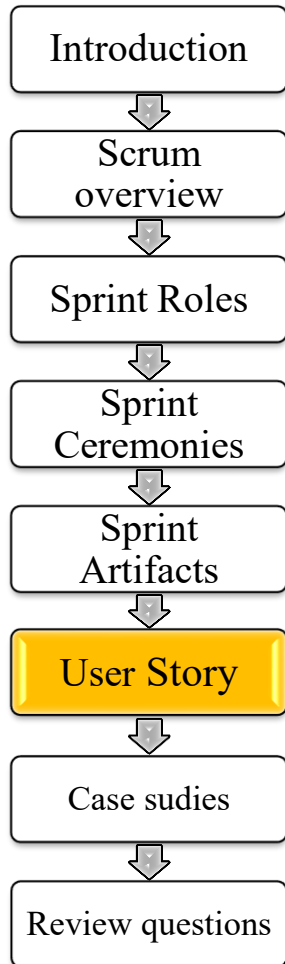
- As a CF requester
- I want to have a checklist of outputs
- So that I can submit a complete CF request

- As an EBC requester
- I want to know what catalog of services TYREE IT offers
- So that I can determine if an existing platform can support my proposed project

- As xxxxx
- I want to xxxxx
- So that xxxxxxx



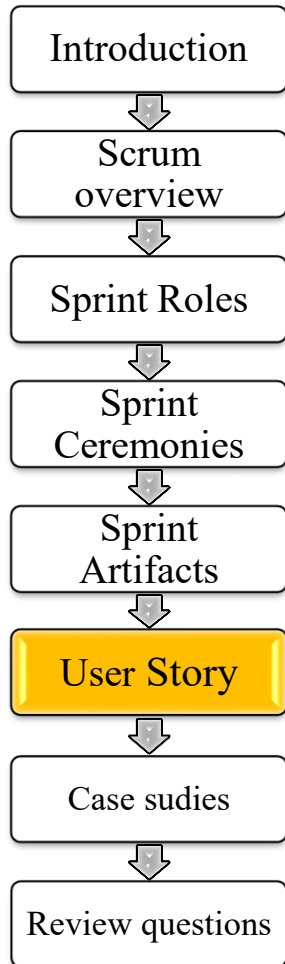
User Stories: hints



- The more detailed a user story, the easier its decomposition in tasks.
- As a logged user
- I want to manage my own mail
- So that I can
 - Organize the mail I received and the mail I sent
 - Read the mail I sent and received
 - Hide mails
 - Compose mails
 - Create, update and delete mail contacts
 - Create, update, delete appointments
 - Retrieve mails by name and/or date

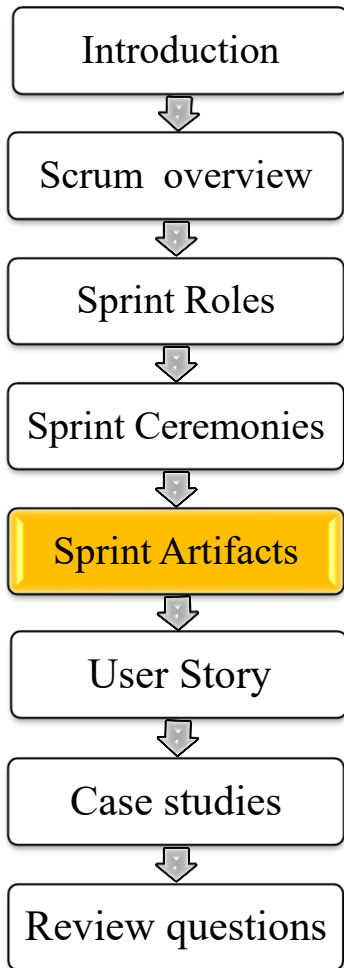


User Stories: hints



- The effort for a user story is predefined, e.g.: 5 people for one month, namely 5 person-month.
- Hence the total effort for software development can be estimated based on the number of user stories.
- E.g.: A Healthcare Reception software requires 5 user stories, hence $5 \times 5 = 25$ person-months.
- To that development effort other costs should be added, such as:
 - Within the software project: user requirements, coordination, test facilities and effort.
 - For the overall project /program: organizational change, infrastructure, etc.
 - PMO (Project Management Office).

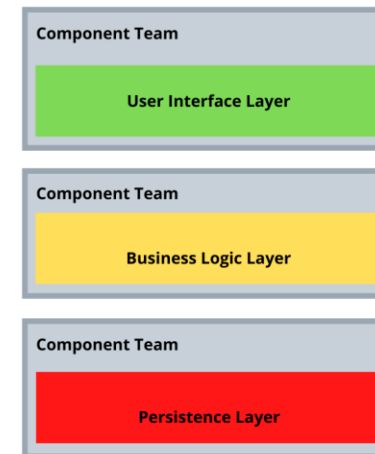
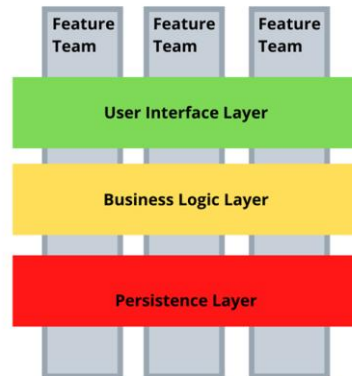
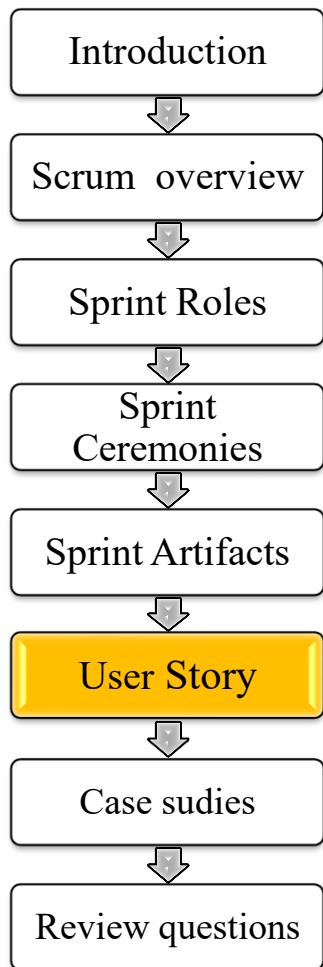
User stories: story points (in the Fibonacci sequence)



What is known about the tasks	All	Almost all	All	Almost nothing	Nothing	Nothing
Dependencies	None	Almost none	Some	Few	More than few	Unknown
How much work effort	Less than 2 hours	Half a day	Up to 2 days	Few days	Around a week	More than a week
Story points	1	2	3	5	8 To be split in smaller items	13 To be split in smaller items

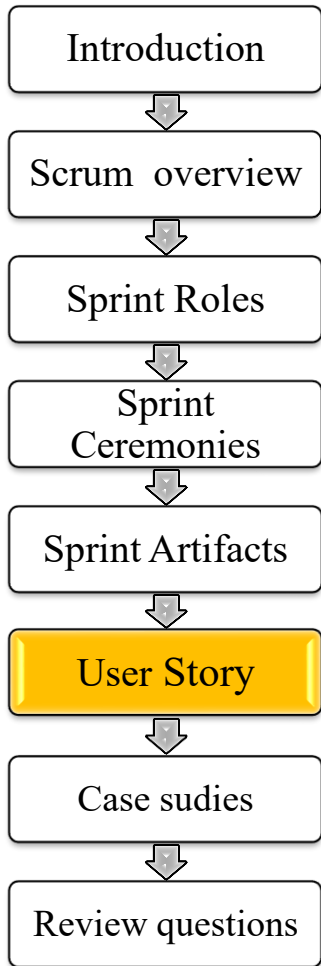


User stories: Component versus Feature Backlog



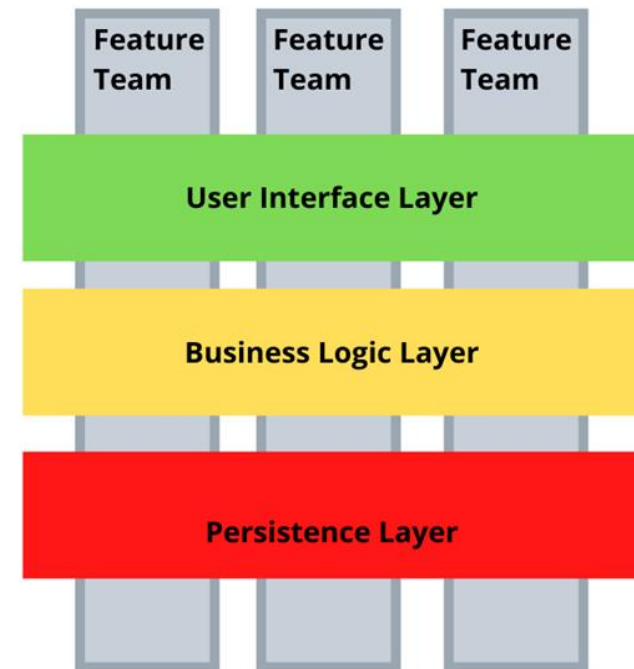
- In general terms, a software app can be decomposed either by layer or feature (function)
- Typical layers are
 - Data Layer (typically database),
 - Business Logic Layer (the logic, backend, microservices),
 - User Interface Layer (the front end).
- Accordingly, development teams can be structured either vertically (feature teams) or horizontally (component teams)

User stories: Component versus Feature Backlog



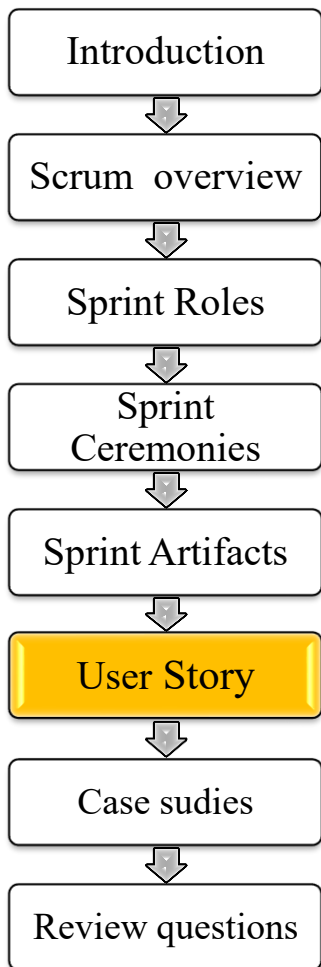
FEATURE TEAMS

- Feature teams are customer-oriented
 - Backlog items in the Sprint Planning are User Stories that allow an immediate benefit from the work done by the feature team during the Sprint or the iteration .
 - Additionally, feature teams have very low coupling
- Feature teams allow faster release and immediate user feedback
- However, feature teams do not focus architecture that could result weak with a low code quality and scalability.



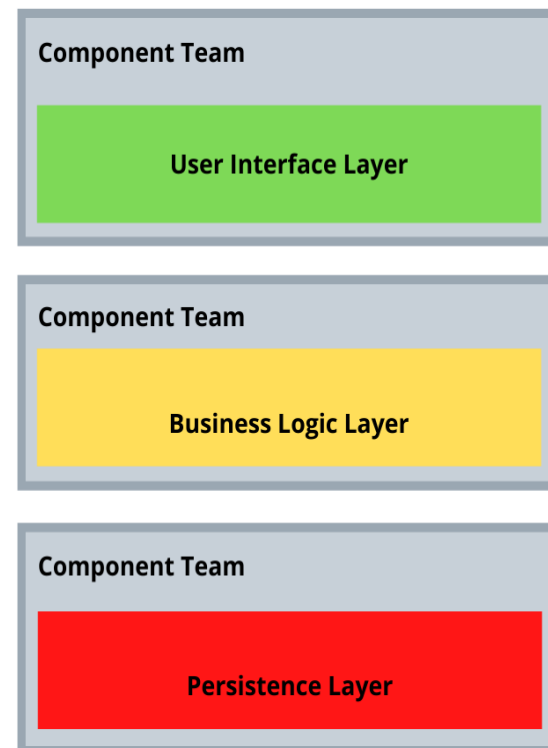


User stories: Component versus Feature Backlog



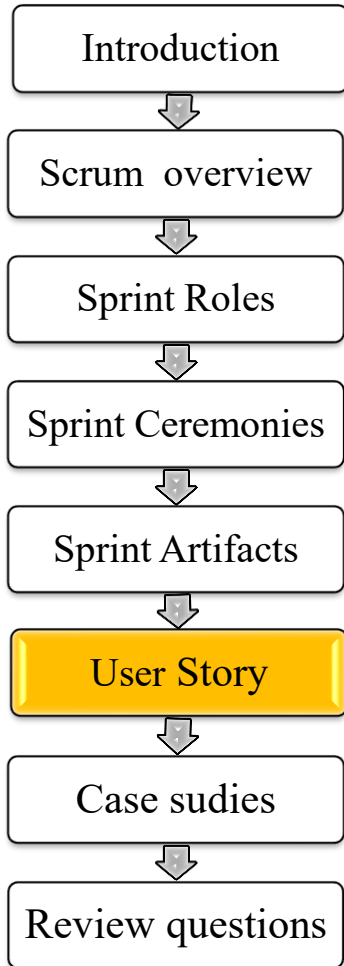
COMPONENT TEAMS

- A component team works on a block of the architecture.
- Coordination of component teams may become complex, because the outputs must be consistent across teams.
- Component teams should translate user requirements into the task – a complex job.
- Component teams may be over-focused on technology, thus neglecting the customer



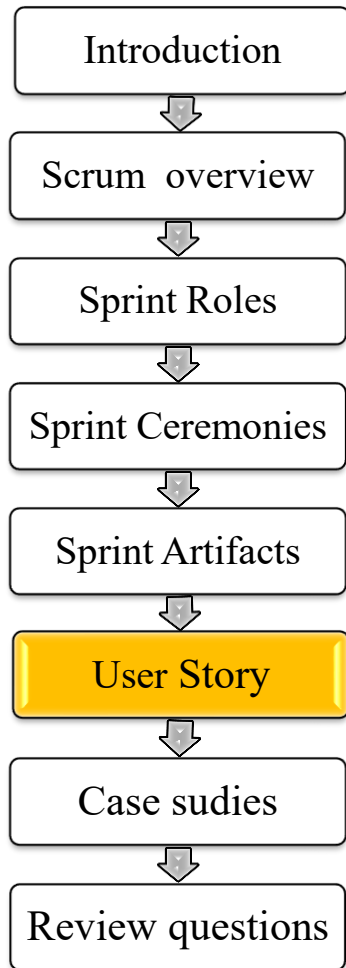


User Story Prioritization



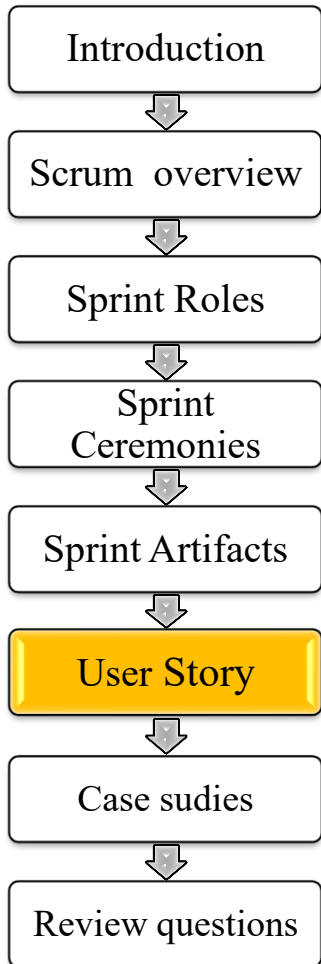
- Scrum and Agile define the templates of
 - Product Backlog
 - Sprint Backlog
 - User Story
- However, they do not provide a specific method for prioritizing or sequencing the User Stories, and, more generally, the functions to be implemented in a backlog.
- “Story mapping” (published in 2014 by Jeff Patton) provides these criteria
- A more general criterion is MVP (Minimum Viable Product)
- ***PS: both prioritizing approaches can be used (and are used) in predictive approaches (waterfall)***

User Story Prioritization: User Story Maps



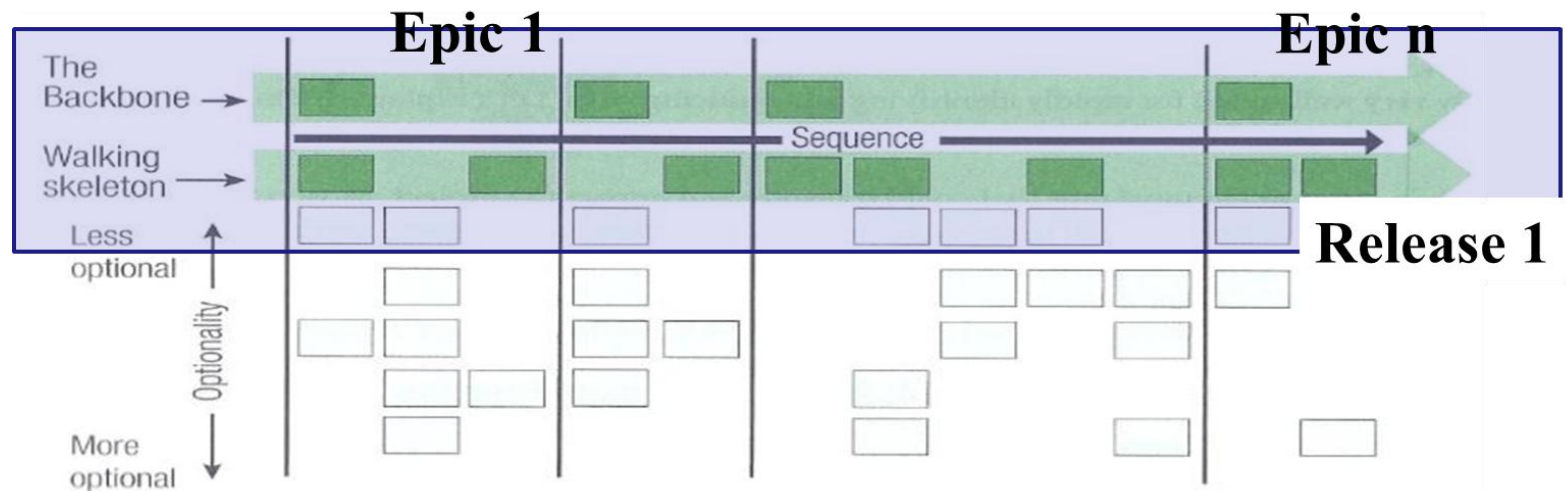
- As Agile Alliance states (<https://www.agilealliance.org/glossary/storymap/>)
 - *Story mapping consists of ordering user stories along two independent dimensions.*
 - *In the horizontal axis stories are roughly in the order in which the user would perform the task.*
 - *In the vertical axis, user stories are ordered by priority and/or increasing sophistication of the implementation.*
 - *In a story map so arranged, the first horizontal row represents a “walking skeleton“, a barebones but usable version of the product.*
 - *Working through successive rows fleshes out the product with additional functionality.*

User Story Prioritization: User Story Map



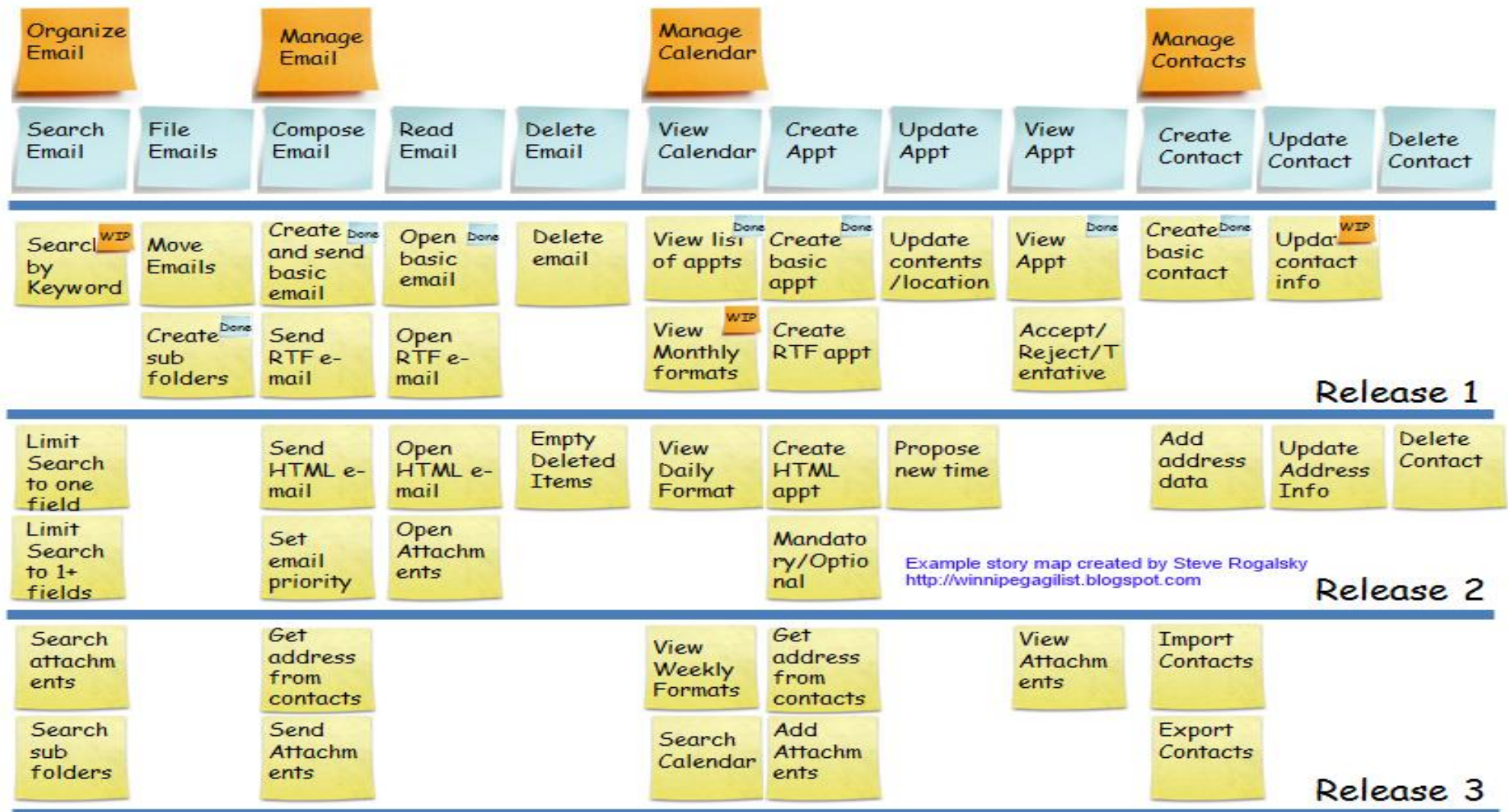
Story Maps help select and group features for a release.

The «Walking Skeleton» (alias MVP) is the smallest system that could work.





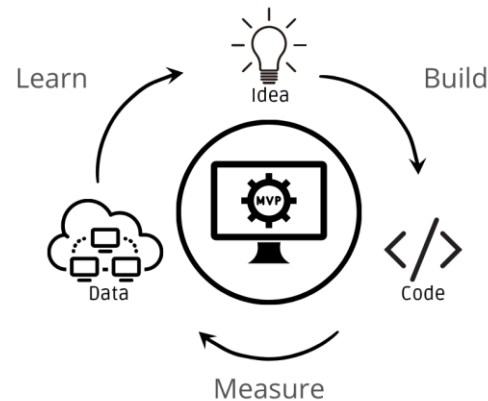
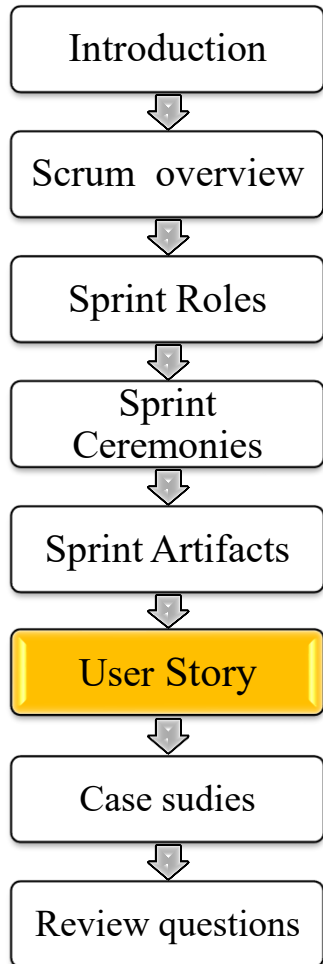
User Story Prioritization: User Story Map: example



User Story Prioritization :

MVP Minimum Viable Product

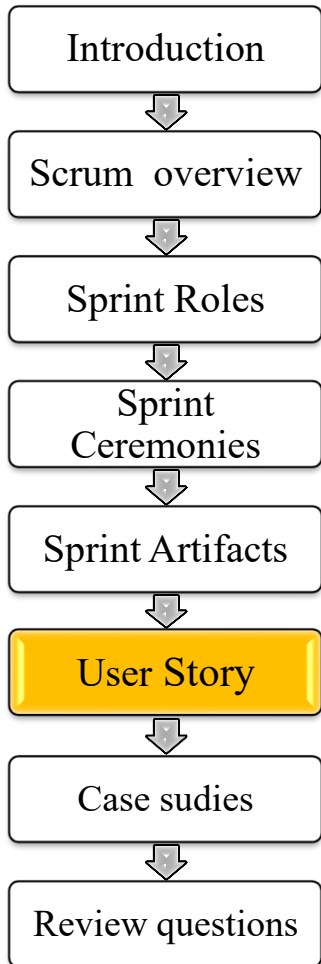
Source <https://www.techopedia.com/definition/27809/minimum-viable-product-mvp>



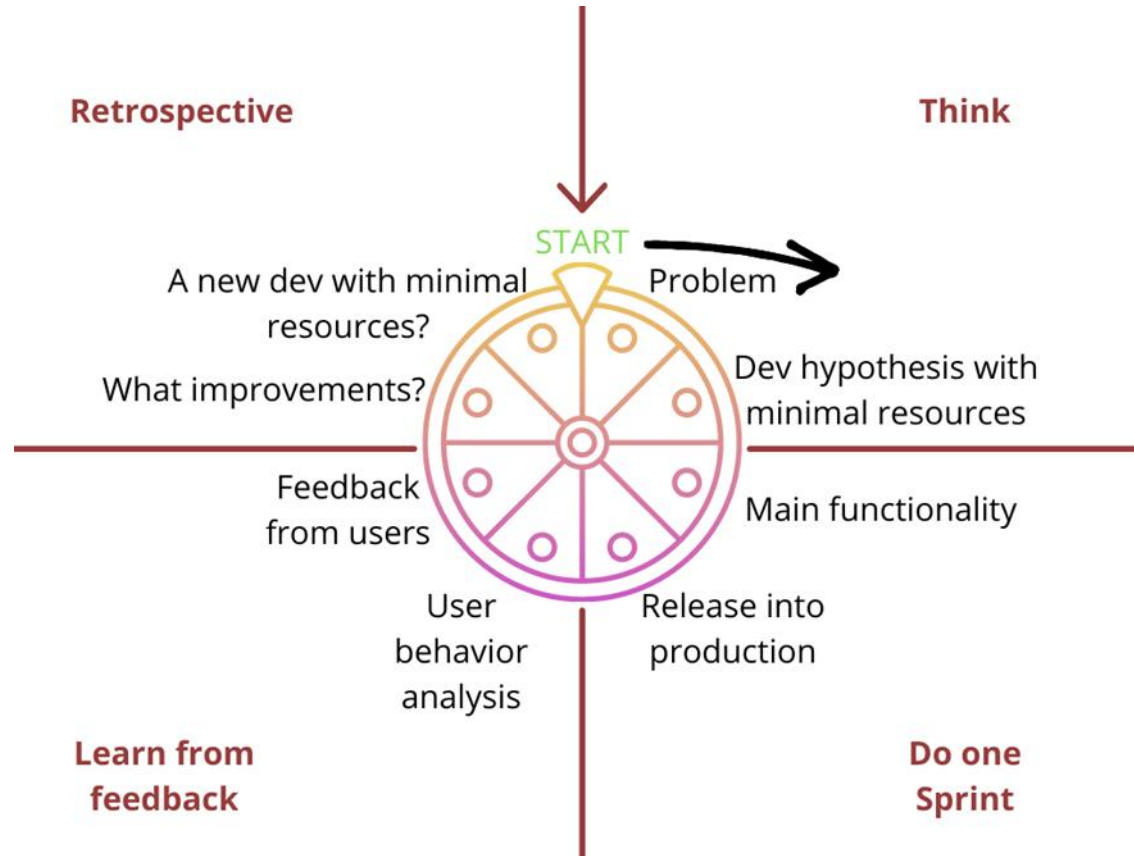
- Minimum Viable Product (MVP), is a term coined and defined in 2001 by Frank Robinson
- A minimum viable product (MVP) has sufficient features to satisfy early adopters.
- The final, complete set of features is only designed and [will be] developed after considering feedback from the product's initial users.

User Story Prioritization : MVP Minimum Viable Product

Source <https://www.techopedia.com/definition/27809/minimum-viable-product-mvp>

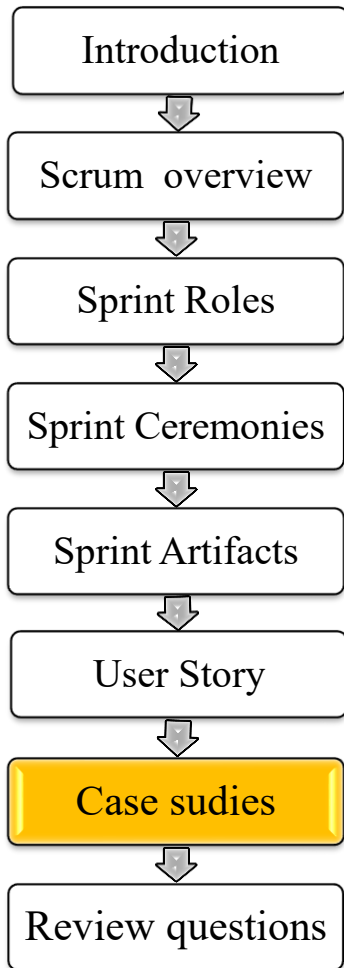


- MVP can be implemented in various ways:
 - Core feature: the product has only one functionality.
 - Video demo: the product is not implemented, but you have a video of the product.
 - Landing page: users "land" on a web page where is the presentation of the product.
 - Wizard of Oz: the system is manual, but the user perceives it is automated
 - Piecemeal: MVP is built using various tools. E.g., Groupon used Apple mail for email management, Wordpress for the website, FileMaker for creating coupons.
 - Concierge: the customer knows that the interaction is human.
 - Crowdfunding: people fund a startup before the product exists.
 - Pitch desk: presentation of a business idea aimed at investors.





FBI sentinel

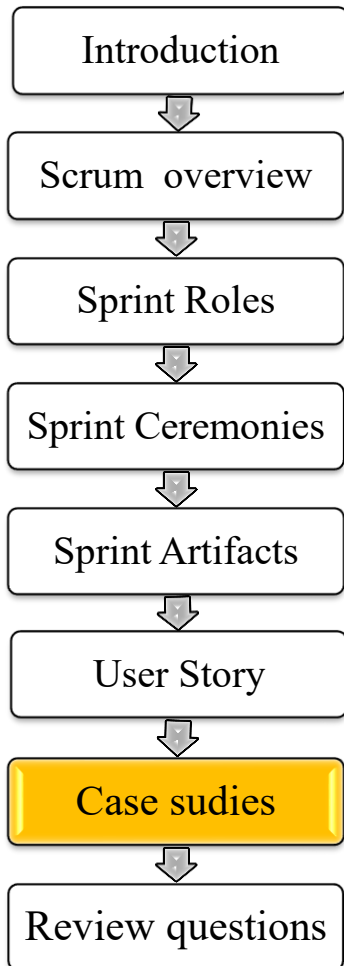


- After September 11th, 2001, FBI launched an anti-terrorism project, named Sentinel.
- In March 2006 FBI awarded a \$ 305 million contract to Lockheed Martin for Sentinel. However, this project failed and was terminated in July 2010.
- In September 2010, FBI decided to complete Sentinel with an Agile methodology

Source: Sutherland, Jeff; Sutherland, J.J. (September 30, 2014). Scrum: The Art of Doing Twice the Work in Half the Time (1st ed.)



FBI sentinel

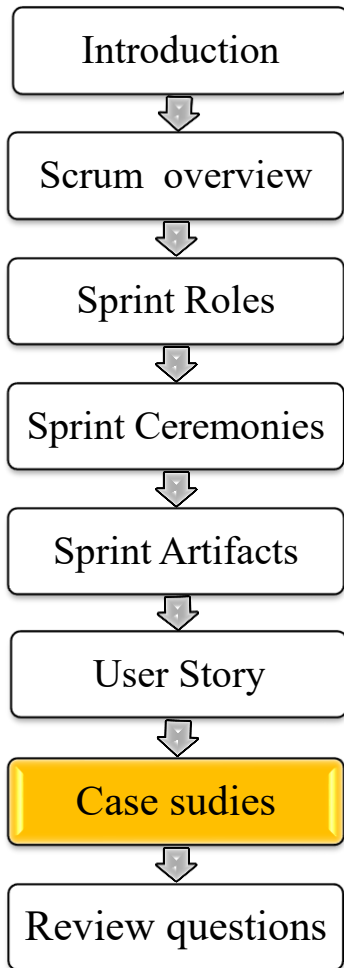


- The project included 670 User Stories The teams self-organized and the project started with
 - 45 staff,
 - the Product Owner managing priorities,
 - two-week Sprints
 - project size: roughly 1,500 person-month
- Sentinel was released in July 2012, 5 months later than expected. But, since two previous attempts were unsuccessful, the 5-month delay was not perceived as a problem.

Source: Sutherland, Jeff; Sutherland, J.J. (September 30, 2014). Scrum: The Art of Doing Twice the Work in Half the Time (1st ed.)



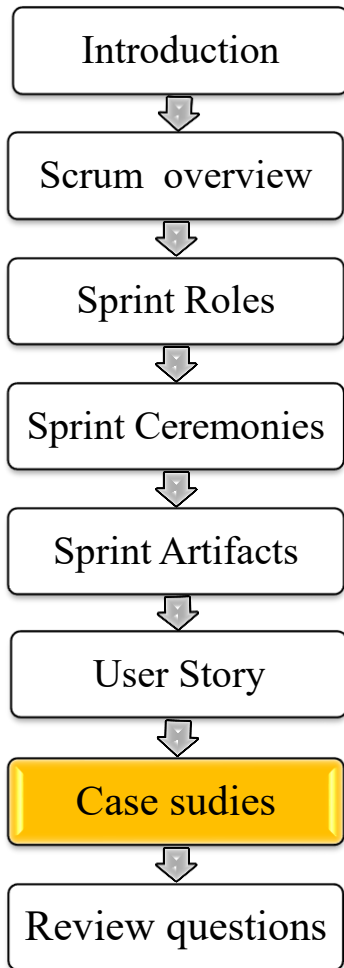
FBI sentinel: comments 1



- The Sentinel project is a relevant case study for Agile, because:
 - complexity of the system: Agile was successful, following two other failed projects, run by traditional project management;
 - size: Agile was successful in a very large project, with a budget of millions of dollars
 - mission critical: the project concerned a critical government mission

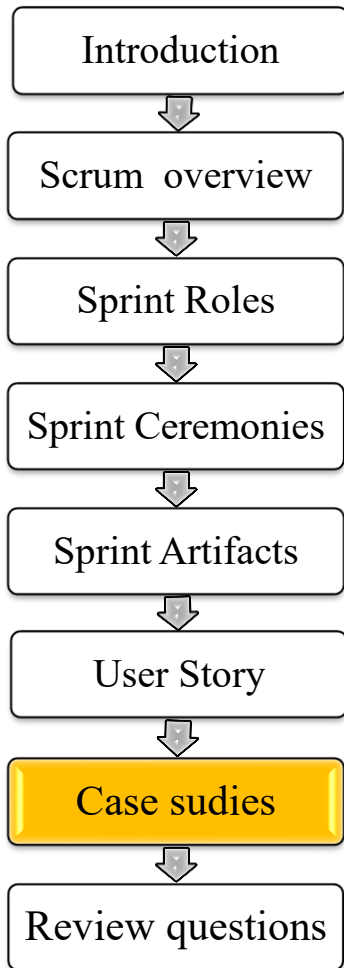


FBI sentinel: comments 2



- Sentinel suffered performance issues at release, both hardware and architectural.
- **In fact, teams did not address some tests, and from the Burn Down Chart perceived that the work was done.**
- **Indeed, performance and integrity are critical in every COMPLEX system**
- The position of the integrity check in the project lifecycle is critical.
 - In some projects, integrity shall be checked during the project, or in the first release stages (pilot installation).
- Performance tests and test scenarios resemble mechanical integrity checks or shake-down practices; for instance, in airspace industry, the prototype plane is flown many times to find out problems

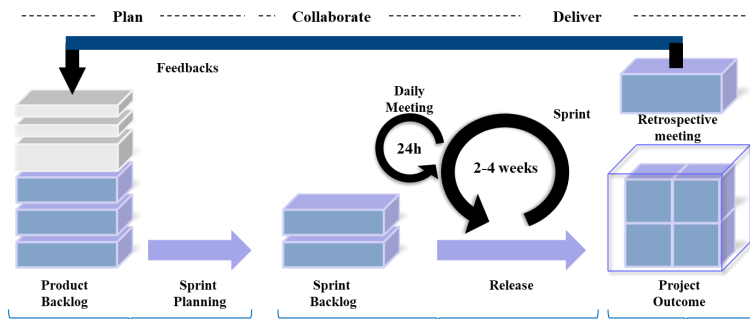
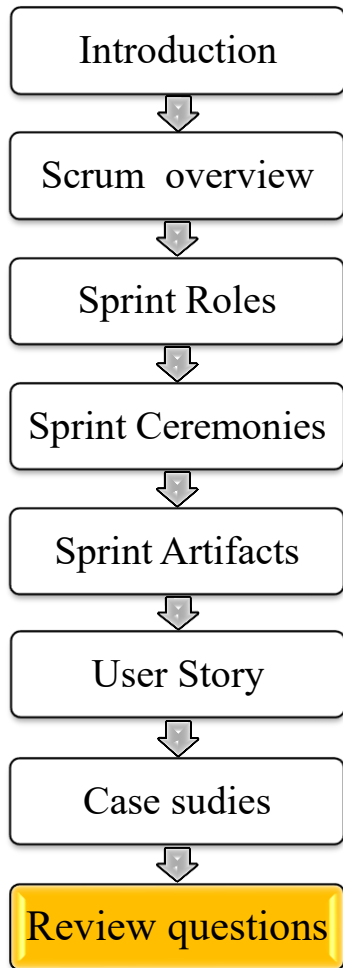
Agile exercise: sketch a Product Backlog and related Story Map



- The university committee needs various documents be prepared for the defense day:
 - 1) copy of the thesis;
 - 2) report of the advisor;
 - 3) report of examiner;
 - 4) student's career log.
- The student needs to submit the PDF file of the thesis 30 days in advance, along with related metadata.
- The advisor accepts the submission and allows the student to attend the session.
- The advisor and the examiner need to fill and submit the report using a predefined form at least 5 days in advance.
- The secretary needs to print the reports to add to the committee documents folder.
- The student receives notification whether his/her submission is approved or rejected with a motivation.



Review Questions: SCRUM framework

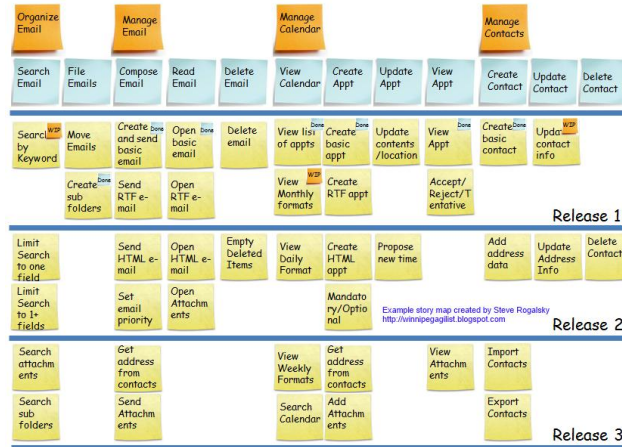
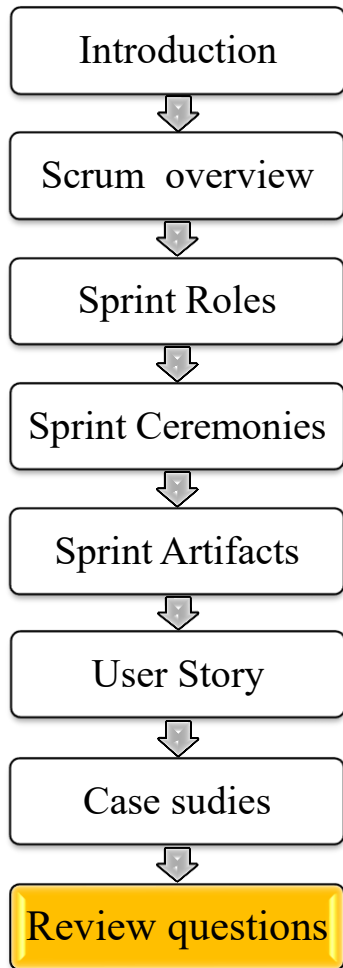


- Please illustrate scope (which projects fit SCRUM) and purpose (what SCRUM is for)

- Please describe framework elements:
 - Product backlog
 - Sprint planning
 - Sprint backlog
 - Daily meeting
 - Retrospective meeting
- Please describe roles
 - PO: Product Owner
 - SM: SCRUM Master
 - ST: SCRUM Team
 - TO: Technical Owner



Review Questions: Agile requirements



- Please explain these Agile concepts:

- Theme
- Epic
- Story
- Task

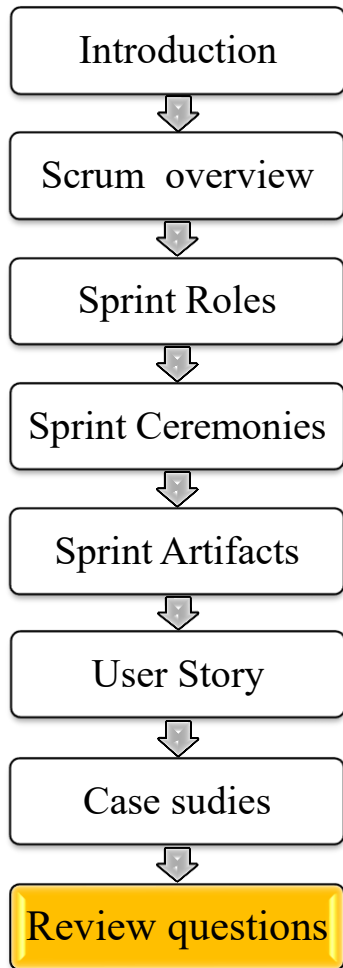
- Please explain and exemplify what is a Story Map

- Please explain these Agile priority concepts:

- MVP
- Walking Skeleton



Agile Exercises



User story 1:

- **As** a student
- **I want** to submit the thesis, a pdf file
- **So that** it is logged in the defense db

Split in tasks:

- setup web application
- put student in like html file
- write a form and upload a file
- persistence layer: File System or DB

User Story 2

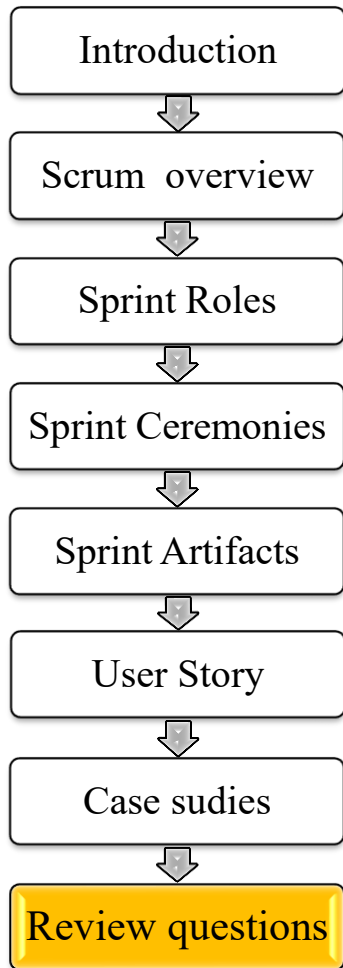
- As an advisor
- I want to be notified
- so that I can accept a report (=status change of the thesis)

Split in tasks:

- The <select> element is used to create a drop-down list.
- Put “student” and “advisor” as elements in the list
- (I want to be notified) up to you, by email or in the webapp, it means creating a profile “advisor”
- “advisor” logs in and sees the document and metadata
- “advisor” can accept the repor



Agile Exercises



User Story 3

- **As** an examiner/supervisor
- **I want** to fill in a validation
- **So that** I add the report to be defense database

Split in tasks:

- profiling: add the element “examiner/supervisor”
- there is a possibility to see the document and metadata
- add a field, text area for validation
- the notes/repost is added in persistence layer, example database

User Story 4

- **As** a secretary
- **I want** to print the complete folder
- **So that** the defense gets the status “ready”

Split in tasks:

- profiling: add the element “secretary”
- make a unique file with all the information, so that the secretary can open it and print it



ADDITIONAL AGILE TECHNIQUES EXTREME PROGRAMMING

R 251003

Contributors (in alphabetical order)

Alessandro Ceseno, Gianmario Motta, Nicola Sfondrini

Editor

Gianmario Motta, motta05@unipv.it



Simple Design

Kent Beck in 1990 developed the 4 rules of Simple Design.

The 4 rules of Simple design are:

- 1. Passes all tests (you DO HAVE tests).
- 2. Reveals intention (good names, clean code), easy to understand.
- 3. No duplication (DRY principle), avoid duplications.
- 4. Fewest elements (KISS principle), minimize elements.

These 4 rules are in priority order, e.g., “Passes all tests” is more important.



Extreme Programming (XP)

- Extreme Programming (XP) is a set of practices for agile development.
- We here illustrate its principles and practices:
 - Fine-scale feedback
 - Continuous process
 - Shared understanding
- References
 - <https://agilealliance.org/glossary/xp/>
 - https://en.wikipedia.org/wiki/Extreme_programming
 - Beck, K. (1999). Extreme Programming Explained: Embrace Change. Addison-Wesley.

Extreme Programming: Fine-scale feedback



- Fine scale feedback relies on:
 - Pair programming:
 - A driver writes the code while a navigator (or observer) reviews the code and sketches the TODO list.
 - The couple can change roles: the driver becomes navigator while the navigator becomes driver
 - On-site customer: the customer attends periodic meetings and physically is in a location close to the development team

Extreme Programming: Continuous process



- Continuous process relies on:
 - Continuous integration: the integration of the code is as frequent as possible (the more frequent the fewer problems)
 - Refactoring: maintaining the same functionality while improving the code, for example by eliminating code duplication
 - Small releases: releasing a few features, for example even one feature at a time, for reducing risks

Extreme Programming: Shared understanding



- Shared understanding relies on:
 - Coding standards: having a common way in which code is written, for example tab with 4 spaces, convention on method names and class names
 - Collective code ownership: every team member is responsible for the whole code, and changes the code developed by other team members as needed
 - Simple design: "simple is the best". The question to be is "what is the simplest thing to get the result?"
 - System metaphor: describing the system as a metaphor



SCALED AGILE

R 250827

Contributors (in alphabetical order)

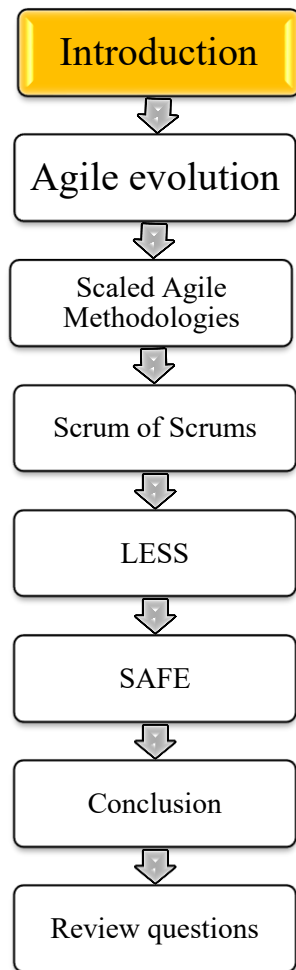
Alessandro Ceseno, Gianmario Motta, Nicola Sfondrini

Editor

Gianmario Motta, motta05@unipv.it



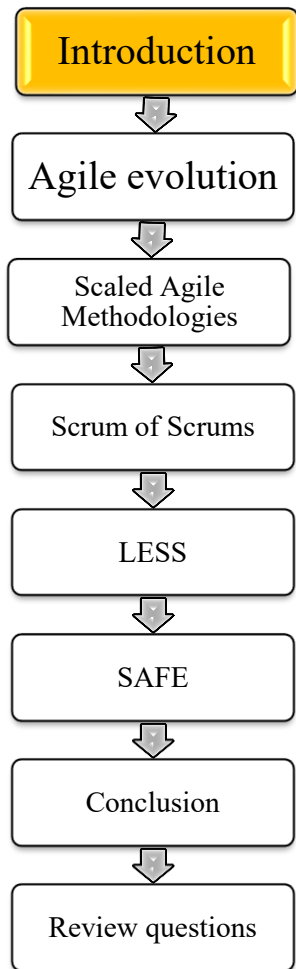
Scaled Agile



- Scrum was conceived for one team
- In large projects multiple teams work in parallel, and, accordingly, an enhanced layer of project management is needed to scale up Agile
- Scaled Agile deals with a threefold challenge
 1. Integrate both definition and refinement of a global backlog, to make sure that the backlogs allocated to each individual team are consistent
 2. Strengthen Sprint planning and control, that should be shared among multiple teams
 3. Keep Scrum ceremonies and Agile approach within each team



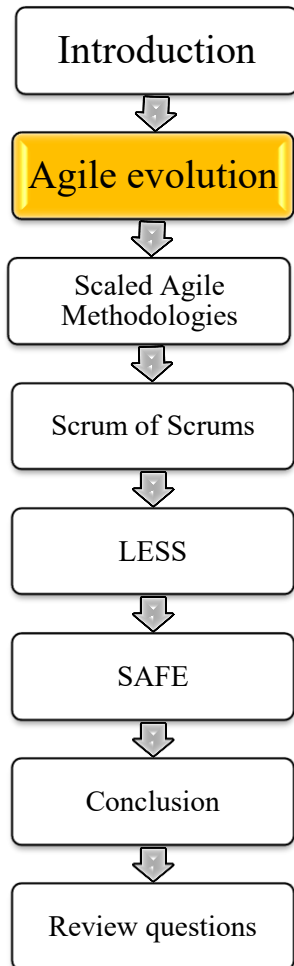
Introduction



- We illustrate:
 - The evolution of Agile methodologies in the scenarios of multiple teams and the related enhancement of Agile practices
 - The resulting multilayered project methodologies, where SCRUM is complemented by Agile project management methodologies



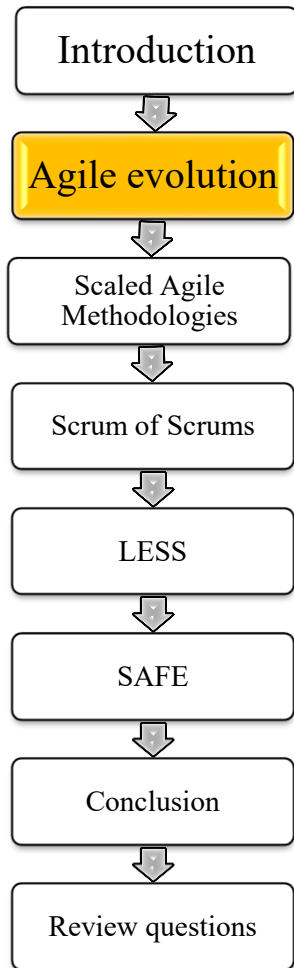
Large projects: enhanced practices



Scrum element	Challenge	Improved Practice
Sprint Planning	Effective Sprint Planning	Multiple local Sprints and one Central Sprint that starts multiple “physical” Sprints in each single location with a physical interaction.
Daily Meeting	Possible lack of team members	Use fixed time box. Split the meeting by location. Do the Daily Scrum.
Review	Make a Review that involves all teams	Prepare a Review agenda. Organize a time slot where the Review is first, then Retrospective and then Planning.
Retrospective	Solve big problems	Scrum masters talk to each other without teams. Use a space where everyone is free to give suggestions and advice. Identify the impediments, prioritize them, seek the cause, find the solution.



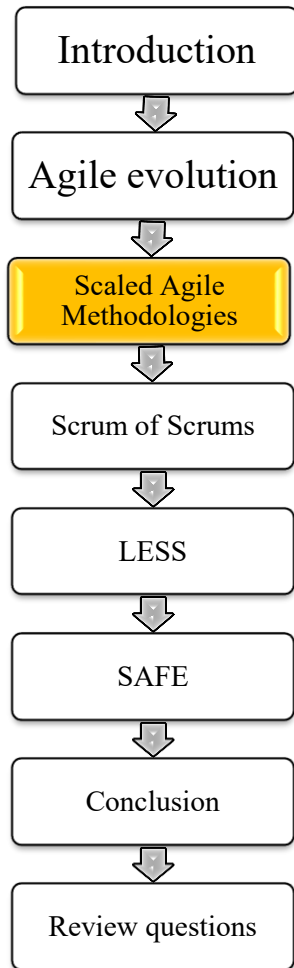
Large projects: enhanced practices



Artifact	Challenge	Improved Practice
Product and Sprint Backlogs	• The user stories of the backlog are edited by different people. • There is a potential technical debt (e.g., the quality of the code gets low with duplications and untested functions)	• Product Owners conduct together Product Backlog Refinements thus allocating a Sprint Backlog to each team. • Each team has its own backlog, where it develops its own set of functions, thus avoiding overlaps. • Heavy investment in informal information channels, such as Instant Messaging.

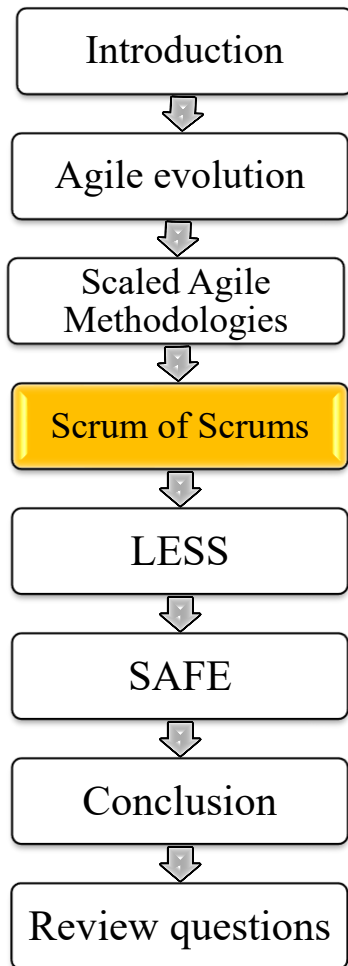


Scaled Agile methodologies



- With the requirement of managing large projects while maintaining Scrum / Agile development approach, Scaled Agile frameworks are growingly complex.
- We here illustrate the main characteristics of:
 - Scrum of Scrums (2001) is just a way of integrating a set of parallel Scrum teams working on the same backlog
 - LeSS (Large-Scale Scrum) organizes a software factory with multiple parallel Scrum teams and one backlog (2005)
 - LeSS Huge, an enhanced version of LeSS
 - SAFe (Scaled Agile Framework) (2007)

Scaled Agile: Scrum of Scrums

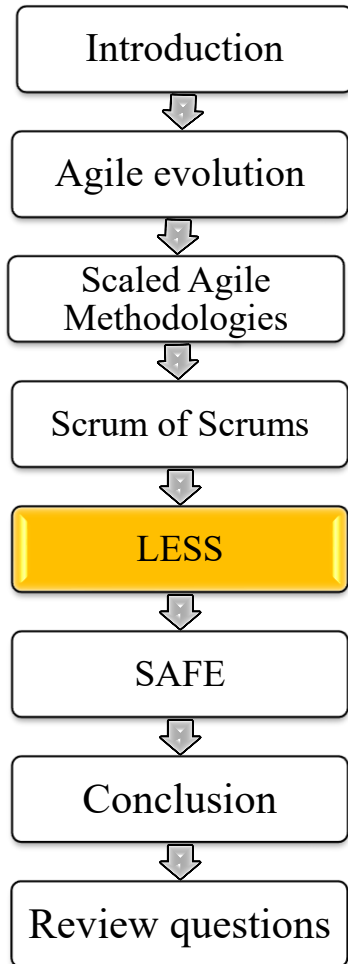


- Scrum of Scrum is a technique to scale_up Scrum to large groups (over a dozen people), by dividing the groups into Agile teams of 5-10.
- Each day a sub-team designates one member as “ambassador” to participate in a daily meeting with the ambassadors from other teams
- Depending on the context, ambassadors may be technical contributors, or Scrum Masters or managers of each team.
- The Scrum of Scrums proceeds as a normal daily meeting, with ambassadors reporting completions, next steps and impediments on behalf of the teams they represent.
- Resolution of impediments focuses on the challenges of coordination between the teams.

Source: <https://www.agilealliance.org/glossary/scrum-of-scrums>

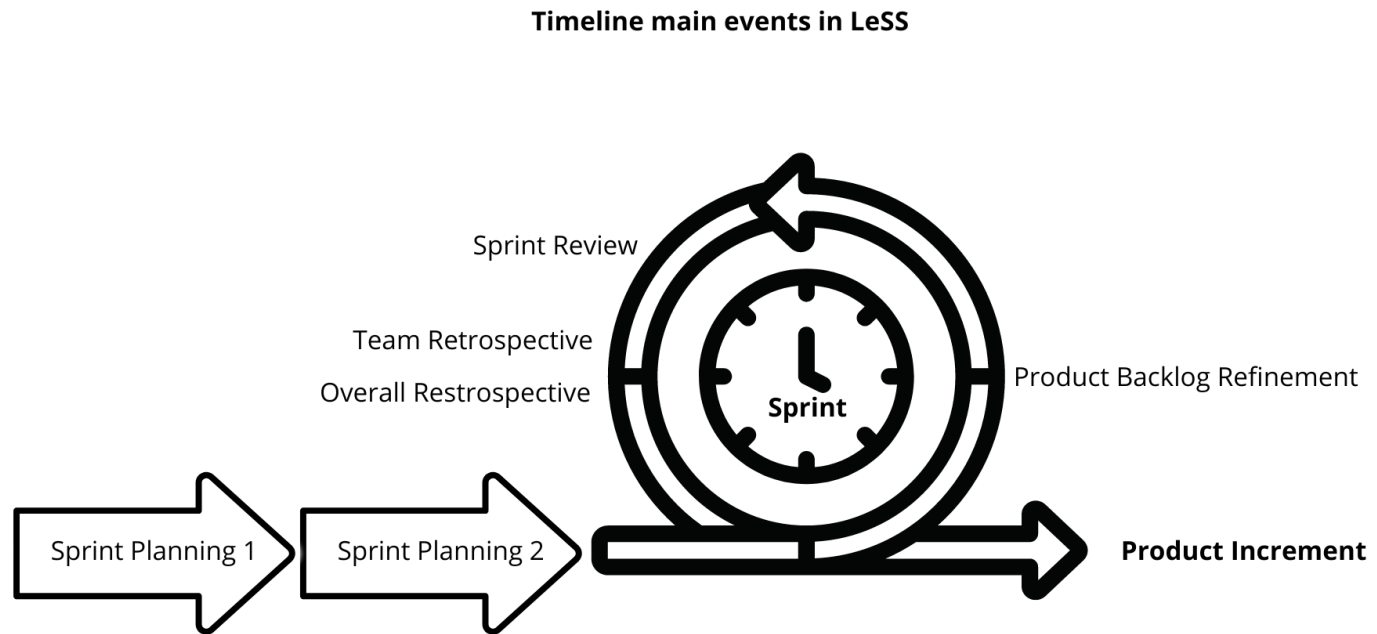
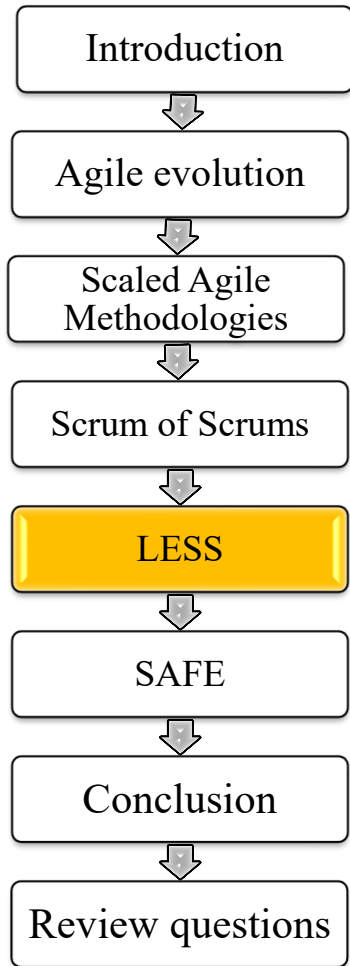
Jeff Sutherland, *Agile Can Scale: Inventing and Reinventing SCRUM in Five Companies* (2001)

Scaled Agile: LeSS (Large-Scale Scrum)

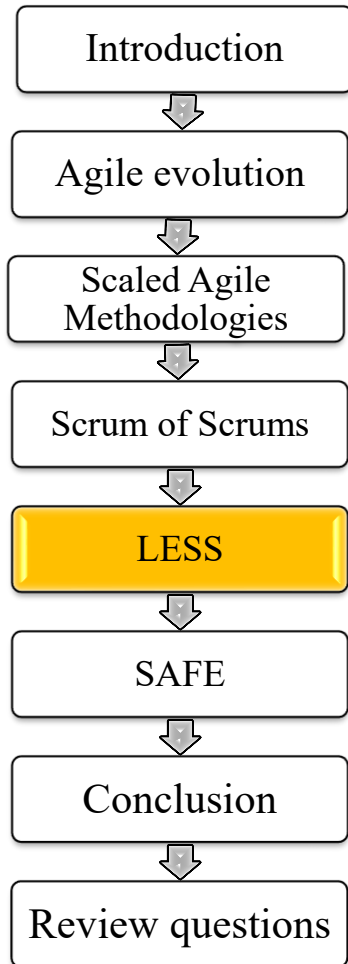


- LeSS is a scaled version of the original Scrum for managing multiple teams that work on the same product
- Hence, LeSS has
 - One Product backlog, because LeSS is for the product and not for the teams
 - One Definition Of Done, i.e., a single DOD for all Teams
 - One Product Owner
 - Multiple "cross-functional teams" compared to "specialist teams"
 - A unique Sprint for all the teams

Scaled Agile: LeSS: overview

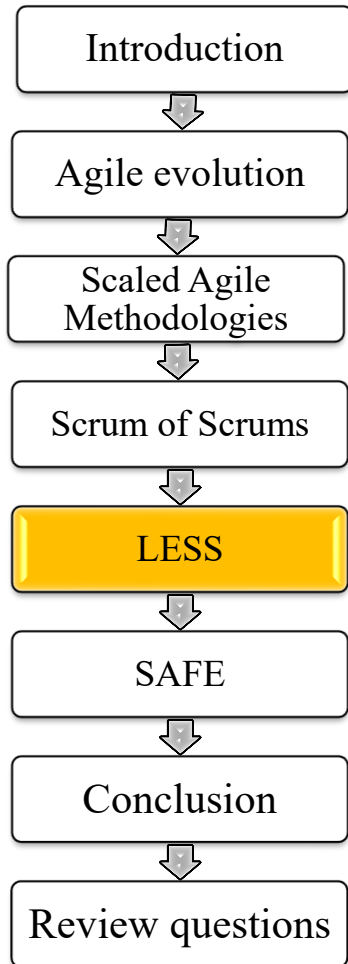


Scaled Agile: LeSS ceremonies



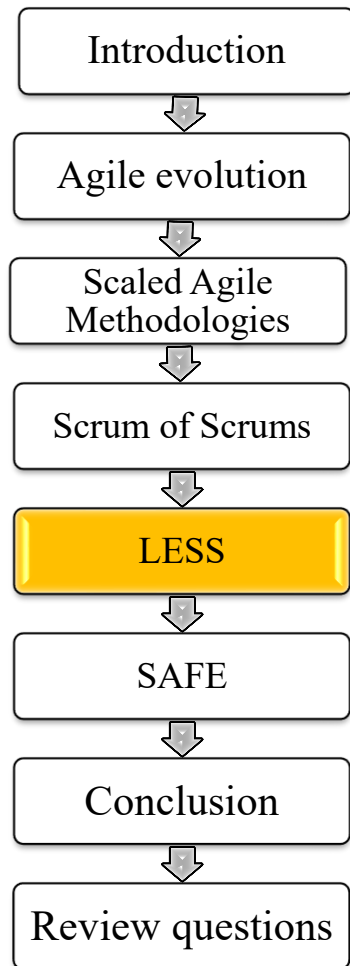
- **Sprint Planning First Part:**
 - With a single product Owner, Sprint Planning involves all the teams, thus deciding the subdivision of the Product Backlog.
 - Teams also discuss opportunities to meet and collaborate for possible developments.
- **Sprint Planning Second Part**
 - It concerns each individual team and involves by exceptions two teams for coordinating interdependent activities
- **Daily Scrum meeting (Stand-up meeting)**
 - Each team has its own Stand-up Meeting. A member of team A can participate as an observer in the Stand-up meeting of team B
- **Coordination**
 - It is based on oral communication and is facilitated by Open Spaces.

Scaled Agile: LeSS Huge

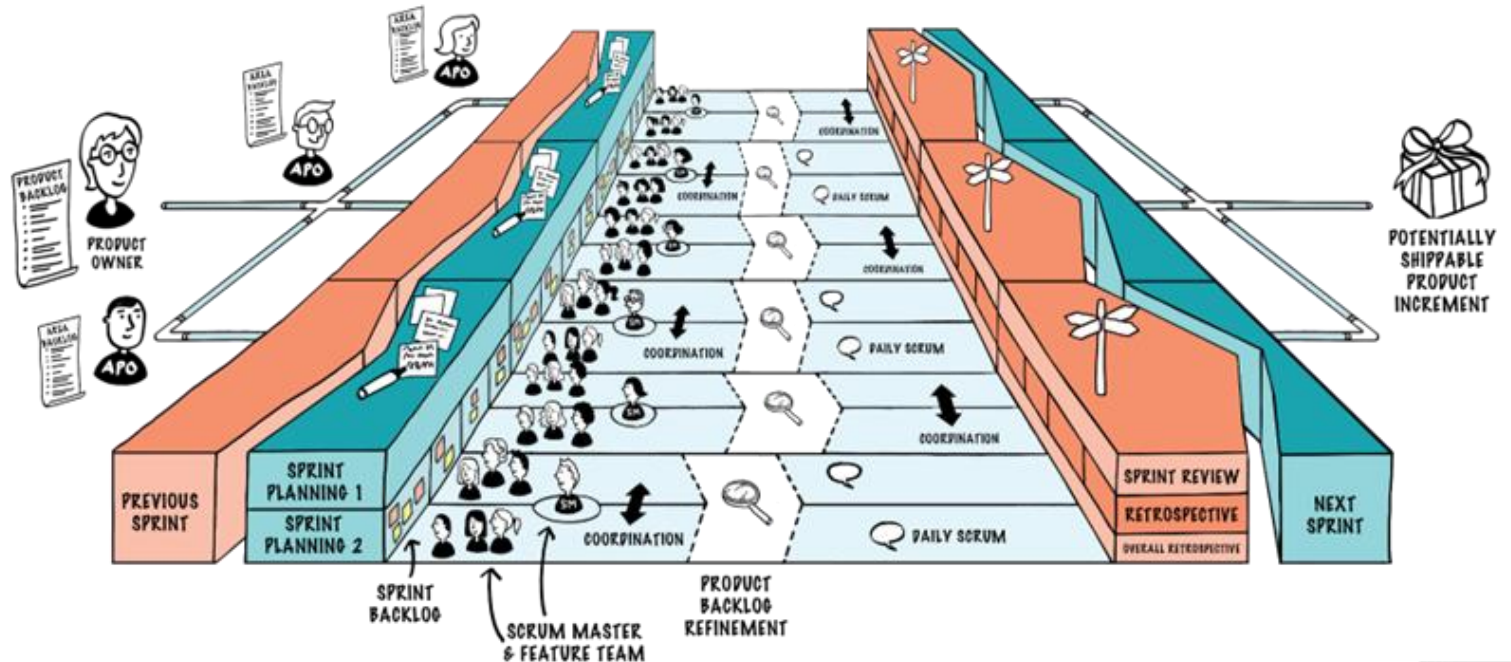


- LeSS Huge framework is for larger organizations (8 teams and more)
- The key concept is the “Requirement Area” (RA), which reflects the interest areas of the customers.
- Each RA includes multiple Scrum teams (e.g., 4) and is managed by its own APO (Area Product Manager) and has
 - A common Backlog
 - A common Planning
 - A common Review
 - A common Overall Retrospective (But a team Retrospective is within each team)
- APOs are further coordinated by a general Product Owner

Scaled Agile: LeSS Huge



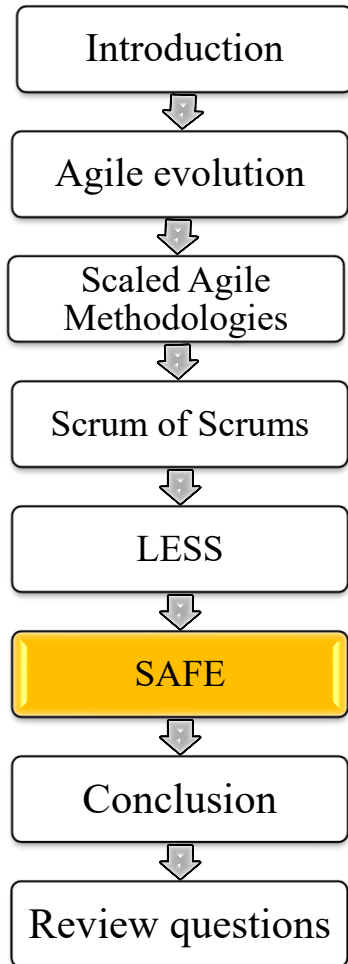
- APO, Area Product Owner
- Product Owner (general)



<http://less.works> BY-ND

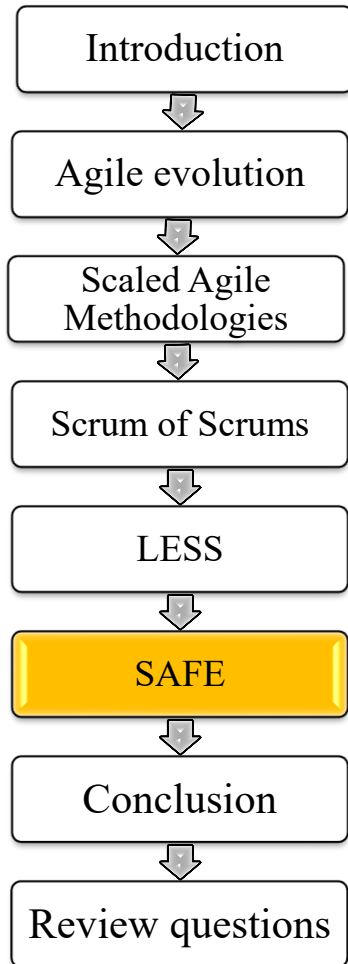
Source: <https://www.cybermedian.com/pl/what-is-less-framework-scrum-vs-less-basic-vs-less-huge/>

Scaled Agile: SAFe (Scaled Agile Framework)



- The Scaled Agile Framework (SAFe) is a set of organization and workflow patterns intended to guide enterprises.
- SAFe is a collection of practices for managing a corporate IT staff, with a much wider scope than LeSS
- SAFe includes various steps that are typical to the IT strategy of a corporation, but it still has Scrum teams
- With the collaboration in the Agile community, it was progressively refined and then formally described in 2007
- 5 major versions have been released, while the latest edition, version 5.1, was released in February 2021

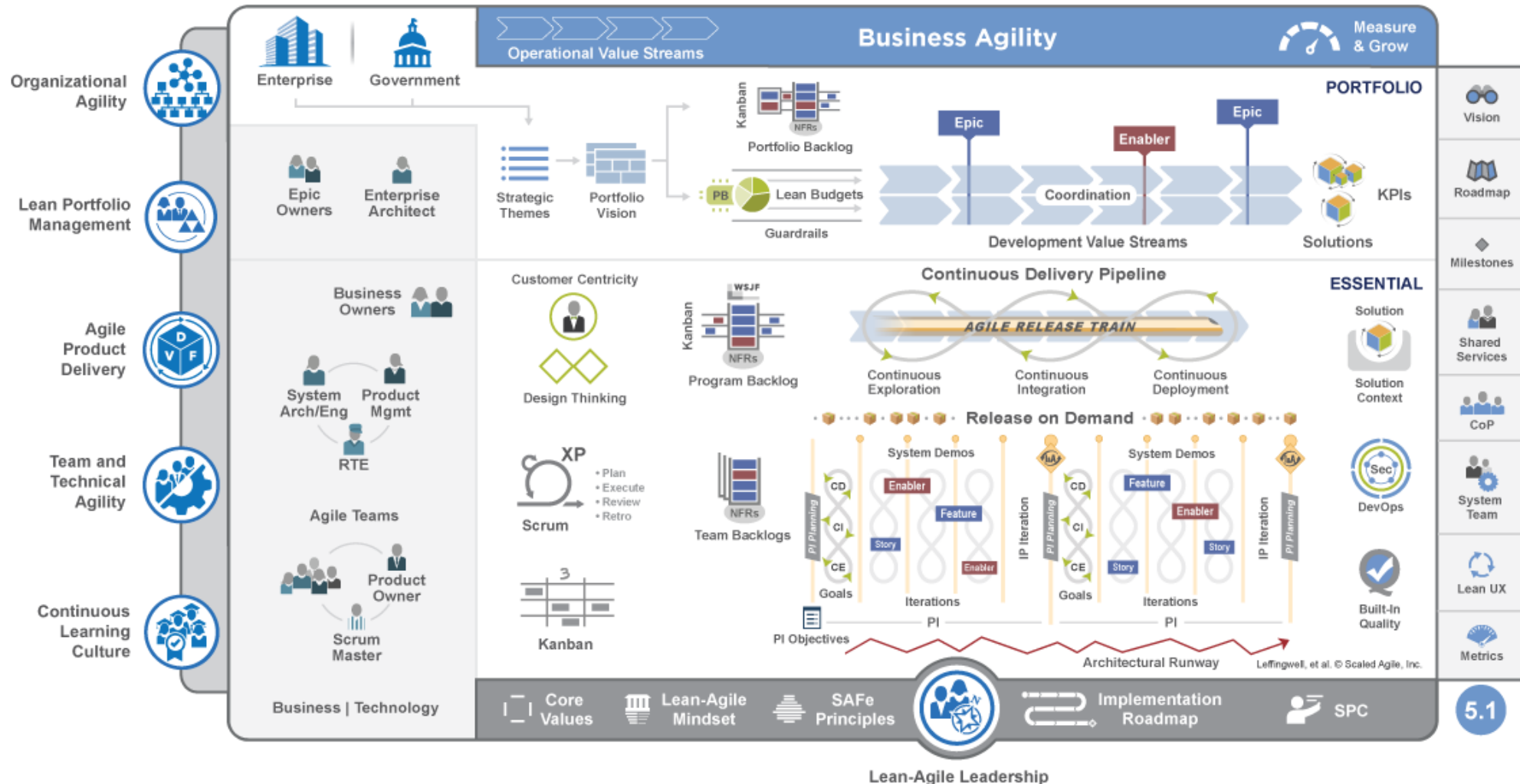
Scaled Agile: SAFe (Scaled Agile Framework)



- SAFe, while it is the most common approach for scaling agile practices, has received criticism for being hierarchical and inflexible
- Given its wide scope and its managerial orientation, SAFe resembles comprehensive IT management frameworks, such as ITIL
 - *ITIL (Information Technology Infrastructure Library) is a set of detailed practices for IT activities such as IT service management (ITSM) and IT asset management (ITAM) that focus on aligning IT services with the needs of business <https://en.wikipedia.org/wiki/ITIL>*

AGILE METHODOLOGIES

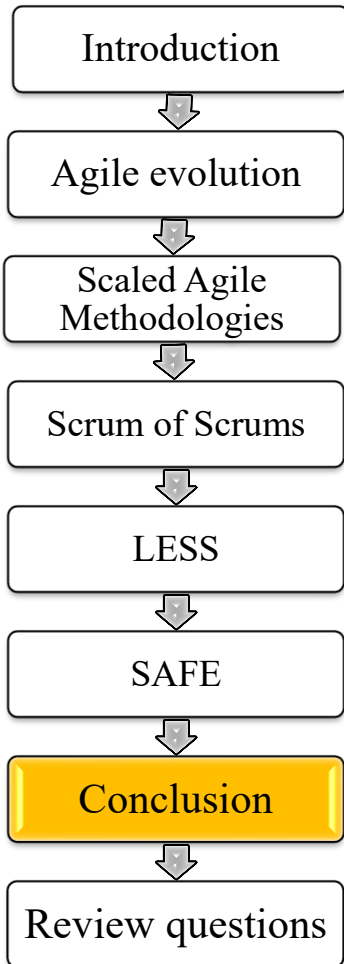
SAFe (Scaled Agile Framework)



Source: <https://www.scaledagileframework.com/>



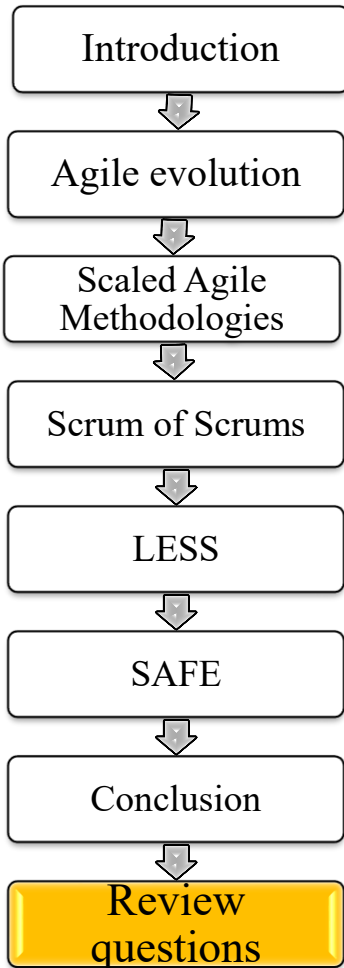
Conclusions: Limits of Scaled Agile



- Scaled Agile methodologies enable organizations to manage Agile projects of any size
- Scaled Agile methodologies are an important addition to the original Agile framework, based on Sprint framework and the User Story template
- However, Scaled Agile frameworks require a high effort for cross-coordination (ambassadors, observers, etc.) thus resembling predictive approaches



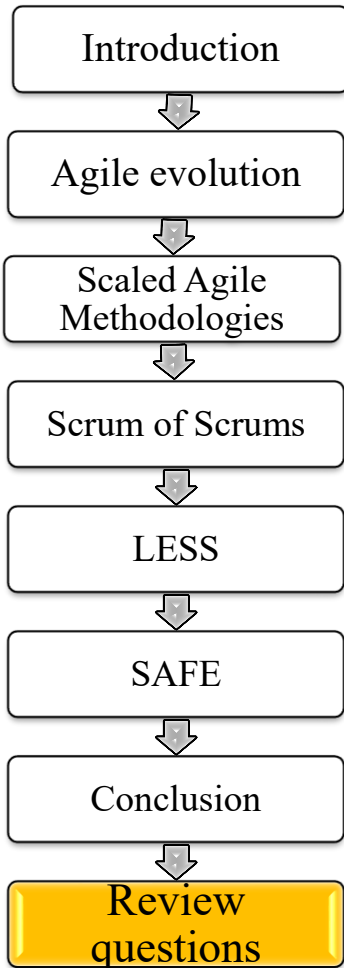
Review Questions



- Explain and comment the challenges of Scaled Agile methodologies
- Explain and comment the project layers (overall project management, teamwork, development/testing methodologies)



Agile in large projects: some references



Using Scrum In Global Software Development” , published in 2009 Fourth IEEE International Conference on Global Software Engineering, Published in the 2009 Fourth IEEE International Conference on Global Software Engineering. IEEE, 2009

“Is Scrum fit for global software engineering”, 2017 IEEE 12th International Conference on Global Software Engineering (ICGSE)

Lase, Kristian Juri Damai, Ridi Ferdiana, and Paulus Insap Santosa. "Optimizing Scrum's Effectiveness in Distributed Software Development Environment during the Pandemic Era: A Systematic Literature Review." 2022 2nd International Conference on Information Technology and Education (ICIT&E). IEEE, 2022.

Kokol, P. (2022). Agile software development in healthcare: a synthetic scoping review. Applied Sciences, 12(19), 9462.

Almeida, F., & Bálint, B. (2024). Approaches for hybrid scaling of agile in the IT industry: A systematic literature review and research agenda. Information, 15(10), 592.